Chapter 5. Type Annotations

Chương 5. Ghi chú về kiểu dữ liệu

Annotating your Python code with type information is an optional step which can be very helpful during development and maintenance of a large project or a library. Static type checkers and lint tools help identify and locate data type mismatches in function arguments and return values. IDEs can use these *type annotations* (also called *type hints*) to improve autocompletion and to provide pop-up documentation. Third-party packages and frameworks can use type annotations to tailor runtime behavior, or to autogenerate code based on type annotations for methods and variables.

Việc ghi chú thông tin về kiểu dữ liệu trong mã Python là một bước không bắt buộc, nhưng lại rất hữu ích trong quá trình phát triển và bảo trì các dự án hoặc thư viện lớn. Các công cụ kiểm tra kiểu dữ liệu tĩnh và các công cụ kiểm tra mã nguồn giúp xác định những trường hợp không phù hợp về kiểu dữ liệu giữa các đối số hàm và giá trị trả về. Các môi trường phát triển tích hợp (IDE) có thể sử dụng các ghi chú về kiểu dữ liệu này để cải thiện chức năng tự động hoàn thành câu lệnh, đồng thời cung cấp thông tin hướng dẫn sử dụng một cách trực quan. Các gói phần mềm và framework của bên thứ ba cũng có thể sử dụng các ghi chú về kiểu dữ liệu để điều chỉnh cách thức hoạt động của chương trình trong quá trình thực thi, hoặc tự động tạo mã nguồn dựa trên các ghi chú này.

Type annotations and checking in Python continue to evolve, and touch on many complicated issues. This chapter covers some of the most common use cases for type annotations; you can find more comprehensive material in the resources listed at the end of the chapter.

Các chức năng liên quan đến ghi chú về kiểu dữ liệu và việc kiểm tra kiểu dữ liệu trong Python vẫn đang tiếp tục được phát triển, và điều này liên quan đến nhiều vấn đề phức tạp. Chương này trình bày một số trường hợp sử dụng phổ biến nhất của các chức năng này. Bạn có thể tìm thêm thông tin chi tiết hơn trong các tài liệu được liệt kê ở cuối chương.

TYPE ANNOTATION SUPPORT VARIES BY PYTHON VERSION

KHẢ NĂNG HỖ TRỢ CÁC LOẠI GHI CHÚ KHÁC NHAU TÙY THUỘC VÀO PHIÊN BẢN CỦA PYTHON.

Python’s features supporting type annotations have evolved from version to version, with some significant additions and deletions. The rest of this chapter will describe the type annotation support in the most recent versions of Python (3.10 and later), with notes to indicate features that might be present or absent in other versions.

Các tính năng của Python liên quan đến việc ghi chú kiểu dữ liệu đã được cải tiến qua từng phiên bản, với nhiều thay đổi quan trọng về mặt thêm hoặc bỏ đi các tính năng này. Phần còn lại của chương này sẽ mô tả các tính năng liên quan đến việc ghi chú kiểu dữ liệu trong các phiên bản Python mới nhất (3.10 và các phiên bản sau đó). Các ghi chú sẽ chỉ ra những tính năng có thể có hoặc không có trong các phiên bản khác của Python.

History Lịch sử

Python is, fundamentally, a *dynamically typed* language. This lets you rapidly develop code by naming and using variables without having to declare them. Dynamic typing allows for flexible coding idioms, generic containers, and polymorphic data handling without requiring explicit definition of interface types or class hierarchies. The downside is that the language offers no help during development in flagging variables of incompatible types being passed to or returned from functions. In place of the development-time compile step that some languages utilize to detect and report data type issues, Python relies on developers to maintain comprehensive unit tests, especially (though far from exclusively![1]) to uncover data type errors by re-creating the runtime environment in a series of test cases.

Về bản chất, Python là ngôn ngữ lập trình có kiểu dữ liệu động. Điều này giúp người lập trình có thể viết mã một cách nhanh chóng, bằng cách đặt tên và sử dụng các biến mà không cần phải khai báo chúng trước. Việc sử dụng kiểu dữ liệu động giúp tạo ra các cách lập trình linh hoạt, các cấu trúc dữ liệu tổng quát, và khả năng xử lý dữ liệu một cách hiệu quả, mà không cần phải định nghĩa rõ ràng các kiểu dữ liệu hay cấu trúc lớp. Tuy nhiên, nhược điểm của Python là ngôn ngữ này không hỗ trợ người lập trình trong việc phát hiện các trường hợp các biến có kiểu dữ liệu không tương thích khi được truyền vào hoặc trả về từ các hàm. Thay vì sử dụng bước biên dịch trong quá trình phát triển để phát hiện các lỗi liên quan đến kiểu dữ liệu, Python yêu cầu người lập trình phải tự kiểm tra kỹ lưỡng các chương trình của mình, đặc biệt là thông qua việc tạo ra các trường hợp kiểm thử để phát hiện các lỗi liên quan đến kiểu dữ liệu.

TYPE ANNOTATIONS ARE NOT ENFORCED

CÁC GHI CHÚ KIỂU NÀY KHÔNG BẮT BUỘC PHẢI ĐƯỢC SỬ DỤNG.

Type annotations are *not* enforced at runtime. Python does not perform any type validation or data conversion based on them; the executable Python code is still responsible for using variables and function arguments properly. However, type annotations must be syntactically correct. A late-imported or dynamically imported module containing an invalid type annotation raises a `SyntaxError` exception in your running Python program, just like any invalid Python statement.

Các ghi chú về kiểu dữ liệu không được kiểm tra trong quá trình thực thi chương trình. Python không thực hiện bất kỳ việc kiểm tra kiểu dữ liệu hay chuyển đổi dữ liệu nào dựa trên các ghi chú này. Chương trình Python vẫn phải tự đảm bảo rằng các biến và tham số được sử dụng một cách chính xác. Tuy nhiên, các ghi chú về kiểu dữ liệu phải tuân thủ các quy tắc ngữ pháp. Nếu một mô-đun được nhập vào sau này hoặc được nhập một cách động chứa các ghi chú về kiểu dữ liệu không hợp lệ, chương trình Python sẽ gặp lỗi, tương tự như khi gặp một câu lệnh Python không hợp lệ.

Historically, the absence of any kind of type checking was often seen as a shortcoming of Python, with some programmers citing this as a reason for choosing other programming languages. However, the community wanted Python to maintain its runtime type freedom, so the logical approach was to add support for static type checks performed at development time by lint-like tools (described further in the following section) and IDEs. Some attempts were made at type checking based on parsing function signatures or docstrings. Guido van Rossum cited several cases on the **Python Developers mailing list** showing that type annotations could be helpful; for example, when maintaining large legacy codebases. With an annotation syntax, development tools could perform static type checks to highlight variable and function usages that conflict with the intended types.

Về mặt lịch sử, việc thiếu bất kỳ loại kiểm tra kiểu dữ liệu nào thường được xem là một điểm yếu của Python, với một số lập trình viên cho rằng đây là lý do để chọn các ngôn ngữ lập trình khác. Tuy nhiên, cộng đồng muốn Python duy trì tự do kiểu dữ liệu chạy, nên cách tiếp cận hợp lý là thêm hỗ trợ kiểm tra kiểu tĩnh được thực hiện trong quá trình phát triển bằng các công cụ giống lint (sẽ được mô tả thêm trong phần sau) và IDE. Một số nỗ lực đã được thực hiện trong việc kiểm tra kiểu dựa trên việc phân tích chữ ký hàm hoặc chuỗi tài liệu. Guido van Rossum đã trích dẫn một số trường hợp trên danh sách thư của Python Developers cho thấy chú thích kiểu có thể hữu ích; ví dụ, khi duy trì các mã nguồn kế thừa lớn. Với cú pháp chú thích, các công cụ phát triển có thể thực hiện kiểm tra kiểu tĩnh để làm nổi bật các cách sử dụng biến và hàm mâu thuẫn với kiểu dự kiến.

The first official version of type annotations used specially formatted comments to indicate variable types and return codes, as defined in **PEP 484**, a provisional PEP for Python 3.5.[2] Using comments allowed for rapid implementation of, and experimentation with, the new typing syntax, without having to modify the Python compiler itself.[3] The third-party package **`mypy`** gained broad acceptance performing static type checking using these comments. With the adoption of **PEP 526** in Python 3.6, type annotations were fully incorporated into the Python language itself, with a supporting `typing` module added to the standard library.

Phiên bản chính thức đầu tiên của chú thích kiểu sử dụng chú thích được định dạng đặc biệt để chỉ ra kiểu biến và mã trả về, như được định nghĩa trong PEP 484, một PEP tạm thời cho Python 3.5. [2] Việc sử dụng chú thích cho phép triển khai nhanh chóng và thử nghiệm cú pháp kiểu mới mà không cần chỉnh sửa trình biên dịch Python. [3] Gói bên thứ ba `mypy` đã được chấp nhận rộng rãi khi thực hiện kiểm tra kiểu tĩnh sử dụng các chú thích này. Với việc áp dụng PEP 526 trong Python 3.6, các chú thích kiểu đã được tích hợp hoàn toàn vào ngôn ngữ Python, với một module hỗ trợ `typing` được thêm vào thư viện tiêu chuẩn.

Type-Checking Utilities Các công cụ kiểm tra kiểu dữ liệu

As type annotations have become an established part of Python, type-checking utilities and IDE plug-ins have also become part of the Python ecosystem.

Khi chú thích kiểu đã trở thành một phần thiết lập của Python, các tiện ích kiểm tra kiểu và plug-in IDE cũng trở thành một phần của hệ sinh thái Python.

mypy

The standalone **`mypy`** utility continues to be a mainstay for static type checking, always up-to-date (give or take a Python version!) with evolving Python type annotation forms. `mypy` is also available as a plug-in for editors including Vim, Emacs, and SublimeText, and for the Atom, PyCharm, and VS Code IDEs. (PyCharm, VS Code, and Wing IDE also incorporate their own type-checking features separate from `mypy`.) The most common command for running `mypy` is simply **`mypy my_python_script.py`**.

Tiện ích độc lập `mypy` vẫn là trụ cột cho việc kiểm tra kiểu tĩnh, luôn được cập nhật (có thể là phiên bản Python!) với các biểu mẫu chú thích kiểu Python đang phát triển. `mypy` cũng có sẵn dưới dạng plug-in cho các trình chỉnh sửa như Vim, Emacs và SublimeText, cũng như cho các IDE Atom, PyCharm và VS Code. (PyCharm, VS Code và Wing IDE cũng tích hợp các tính năng kiểm tra kiểu riêng biệt với `mypy` .) Lệnh phổ biến nhất để chạy `mypy` đơn giản là **`mypy my_python_script.py`** .

You can find more detailed usage examples and command-line options in the `mypy` online documentation, as well as a cheat sheet that serves as a handy reference. Code examples later in this section will include example `mypy` error messages to illustrate the kinds of Python errors that can be caught using type checking.

Bạn có thể tìm các ví dụ sử dụng chi tiết hơn và các tùy chọn dòng lệnh trong tài liệu trực tuyến `mypy` , cũng như một bảng tóm tắt giúp bạn tham khảo tiện lợi. Các ví dụ mã sau trong phần này sẽ bao gồm ví dụ thông báo lỗi `mypy` để minh họa các loại lỗi Python có thể được phát hiện bằng cách kiểm tra kiểu.

Other Type Checkers Các loại Checker khác

Other type checkers to consider using include:

Các công cụ kiểm tra kiểu khác nên cân nhắc sử dụng bao gồm:

MonkeyType

Instagram's **MonkeyType** uses the `sys.setprofile` hook to detect types dynamically at runtime; like `pytype` (see below), it can also generate a *.pyi* (stub) file instead of, or in addition to, inserting type annotations in the Python code file itself.

MonkeyType của Instagram sử dụng móc `sys.setprofile` để phát hiện các loại một cách động khi chạy; giống như `pytype` (xem bên dưới), nó cũng có thể tạo một tệp .pyi (stub) thay vì hoặc bổ sung cho việc chèn chú thích kiểu trong tệp mã Python.

`pydantic`

`pydantic` also works at runtime, but it does not generate stubs or insert type annotations; rather, its primary goal is to parse inputs and ensure that Python code receives clean data. As described in the **online docs**, it also allows you to extend its validation features for your own environment. See **"FastAPI"** for a simple example.

`pydantic` cũng hoạt động khi chạy, nhưng không tạo stub hoặc chú thích kiểu chèn; thay vào đó, mục tiêu chính của nó là phân tích các đầu vào và đảm bảo mã Python nhận được dữ liệu sạch. Như đã mô tả trong tài liệu trực tuyến, nó cũng cho phép bạn mở rộng các tính năng xác thực cho môi trường của riêng bạn. Xem "FastAPI" để có một ví dụ đơn giản.

Pylance

Pylance is a type checking module primarily meant to embed Pyright (see below) into VS Code.

Pylance là một mô-đun kiểm tra kiểu chủ yếu nhằm nhúng Pyright (xem bên dưới) vào mã VS.

Pyre giàn thiêu

Facebook's **Pyre** can also generate *.pyi* files. It currently does not run on Windows, unless you have the **Windows Subsystem for Linux (WSL)** installed.

Pyre của Facebook cũng có thể tạo ra các tệp có định dạng .pyi. Hiện tại, công cụ này không hoạt động trên hệ điều hành Windows, trừ khi bạn đã cài đặt Windows Subsystem for Linux (WSL).

Pyright Bản quyền

Pyright is Microsoft's static type checking tool, available as a command-line utility and a VS Code extension.

Pyright là công cụ kiểm tra kiểu dữ liệu tĩnh của Microsoft. Công cụ này có sẵn dưới dạng tiện ích dòng lệnh và tiện ích mở rộng cho VS Code.

`pytype`

`pytype` from Google is a static type checker that focuses on *type inferencing* (and offers advice even in the absence of type hints) in addition to type annotations. Type inferencing offers a powerful capability for detecting type errors even in code without annotations. `pytype` can also generate *.pyi* files and merge stub files back into *.py* sources (the most recent versions of `mypy` are following suit on this). Currently, `pytype` does not run on Windows unless you first install **WSL**.

`pytype` của Google là công cụ kiểm tra kiểu dữ liệu tĩnh. Ngoài việc xác định kiểu dữ liệu dựa trên các ghi chú về kiểu dữ liệu, công cụ này còn có khả năng phát hiện các lỗi liên quan đến kiểu dữ liệu ngay cả trong những đoạn mã không có ghi chú nào về kiểu dữ liệu. `pytype` cũng có thể tạo ra các tệp tin .pyi và nhập chúng trở lại vào các tệp nguồn .py (các phiên bản mới nhất của `mypy` cũng hỗ trợ tính năng này). Hiện tại, `pytype` không thể chạy trên Windows, trừ khi bạn cài đặt WSL trước.

The emergence of type-checking applications from multiple major software organizations is a testimonial to the widespread interest in the Python developer community in using type annotations.

Sự xuất hiện của các ứng dụng kiểm tra kiểu dữ liệu từ nhiều tổ chức phát triển phần mềm lớn chính là minh chứng cho sự quan tâm rộng rãi của cộng đồng lập trình viên Python đối với việc sử dụng các chú thích về kiểu dữ liệu trong mã nguồn.

Type Annotation Syntax Cú pháp ghi chú kiểu dữ liệu

A *type annotation* is specified in Python using the form:

Ở Python, việc ghi chú các kiểu dữ liệu được thực hiện theo định dạng sau:

Explain this code Hãy giải thích đoạn mã này. Translate into Dịch sang tiếng Việt ▾

```
identifier: type_specification
```

`type_specification` can be any Python expression, but usually involves one or more built-in types (for

example, just mentioning a Python type is a perfectly valid expression) and/or attributes imported from the `typing` module (discussed in the following section). The typical form is:

`type_specification` có thể là bất kỳ biểu thức nào trong Python. Thông thường, *`type_specification`* bao gồm một hoặc nhiều kiểu dữ liệu có sẵn trong Python (ví dụ: việc chỉ đơn giản ghi tên một kiểu dữ liệu trong Python cũng đã là một biểu thức hợp lệ). Ngoài ra, *`type_specification`* cũng có thể chứa các thuộc tính được nhập từ mô-đun `typing` (sẽ được trình bày chi tiết ở phần sau). Dạng thông thường của *`type_specification`* là như sau:

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt▾

```
type_specifier[type_parameter, ...]
```

Here are some examples of type expressions used as type annotations for a variable:

Dưới đây là một số ví dụ về cách sử dụng các biểu thức kiểu dữ liệu như các ghi chú về kiểu dữ liệu của một biến:

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt▾

```
import typing

# an int
count: int

# a list of ints, with a default value
counts: list[int] = []

# a dict with str keys, values are tuples containing 2 ints and a str
employee_data: dict[str, tuple[int, int, str]]

# a callable taking a single str or bytes argument and returning a bool
str_predicate_function: typing.Callable[[str | bytes], bool]

# a dict with str keys, whose values are functions that take and return
# an int
str_function_map: dict[str, typing.Callable[[int], int]] = {
    'square': lambda x: x * x,
    'cube': lambda x: x * x * x,
}
```

Note that **`lambda`**s do *not* accept type annotations.

Lưu ý rằng **`lambda`** s không chấp nhận các ghi chú về kiểu dữ liệu.

TYPING SYNTAX CHANGES IN PYTHON 3.9 AND 3.10

NHỮNG THAY ĐỔI VỀ CÚ PHÁP GÕ CHỮ TRONG PYTHON 3.9 VÀ 3.10

One of the most significant changes in type annotations during the span of Python versions covered in this book was the support added in Python 3.9 for using built-in Python types, as shown in these examples.

Một trong những thay đổi quan trọng nhất liên quan đến cách ghi chú các kiểu dữ liệu trong các phiên bản Python được đề cập trong cuốn sách này là việc Python 3.9 hỗ trợ việc sử dụng các kiểu dữ liệu có sẵn trong Python, như được minh họa trong các ví dụ dưới đây.

-3.9 Prior to Python 3.9, these annotations required the use of type names imported from the `typing` module, such as `Dict`, `List`, `Tuple`, etc.

Trước phiên bản Python 3.9, các ghi chú kiểu dữ liệu này đòi hỏi phải sử dụng tên các kiểu dữ liệu được nhập từ mô-đun `typing` . Ví dụ: `Dict` , `List` , `Tuple` ...

3.10+ Python 3.10 added support for using | to indicate alternative types, as a more readable, concise alternative to the `Union[atype, btype, ...]` notation. The | operator can also be used to replace `Optional[atype]` with `atype | None`.

3.10+ Trong Python 3.10, tính năng sử dụng ký hiệu | để chỉ các kiểu dữ liệu thay thế được bổ sung. Đây là cách diễn đạt dễ hiểu và ngắn gọn hơn so với cách sử dụng các ký hiệu `Union[ atype , btype , ...]` . Ngoài ra, toán tử | cũng có thể được dùng để thay thế các ký hiệu `Optional[ atype ]` bằng `atype | None` .

For instance, the previous `str_predicate_function` definition would take one of the following forms, depending on your version of Python:

Ví dụ, định nghĩa của `str_predicate_function` ở trên sẽ có một trong các dạng sau, tùy thuộc vào phiên bản Python mà bạn đang sử dụng:

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt▾

```
# prior to 3.10, specifying alternative types
# requires use of the Union type
from typing import Callable, Union
str_predicate_function: Callable[Union[str, bytes], bool]

# prior to 3.9, built-ins such as list, tuple, dict,
# set, etc. required types imported from the typing
# module
```

```
from typing import Dict, Tuple, Callable, Union
employee_data: Dict[str, Tuple[int, int, str]]
str_predicate_function: Callable[Union[str, bytes], bool]
```

To annotate a function with a return type, use the form:

Để ghi chú về kiểu dữ liệu được trả về bởi một hàm, hãy sử dụng cú pháp sau:

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
def identifier(argument, ...) -> type_specification :
```

where each *`argument`* takes the form:

Ở đây, mỗi *`argument`* có dạng như sau:

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
identifier[: type_specification[ = default_value]]
```

Here’s an example of an annotated function:

Đây là ví dụ về một hàm được ghi chú chi tiết:

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
def pad(a: list[str], min_len: int = 1, padstr: str = ' ') -> list[str]:
    """Given a list of strings and a minimum length, return a copy of
       the list extended with "padding" strings to be at least the
       minimum length.
    """
    return a + ([padstr] * (min_len - len(a)))
```

Note that when an annotated parameter has a default value, PEP 8 recommends using spaces around the equals sign.

Lưu ý rằng, khi một tham số có giá trị mặc định, theo khuyến nghị của PEP 8, cần sử dụng dấu cách xung quanh dấu bằng.

FORWARD-REFERENCING TYPES THAT ARE NOT YET FULLY DEFINED

CÁC KIỂU DỮ LIỆU THAM CHIẾU NGƯỢC MÀ VẪN CHƯA ĐƯỢC ĐỊNH NGHĨA ĐẦY ĐỦ

At times, a function or variable definition needs to reference a type that has not yet been defined. This is quite common in class methods, or methods that must define arguments or return values of the type of the current class. Those function signatures are parsed at compile time, and at that point the type is not yet defined. For example, this `classmethod` fails to compile:

Đôi khi, việc định nghĩa một hàm hoặc biến đòi hỏi phải tham chiếu đến một kiểu dữ liệu chưa được định nghĩa trước đó. Điều này khá phổ biến trong các phương thức thuộc lớp, hoặc những phương thức cần định nghĩa các tham số hoặc giá trị trả về thuộc kiểu dữ liệu của chính lớp đó. Các thông tin về kiểu dữ liệu này sẽ được xử lý vào lúc biên dịch; tuy nhiên, vào lúc đó, kiểu dữ liệu đó vẫn chưa được định nghĩa. Ví dụ, đoạn mã `classmethod` sẽ không thể được biên dịch:

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
class A:
    @classmethod
    def factory_method(cls) -> A:
        # ... method body goes here ...
```

Since class `A` has not yet been defined when Python compiles `factory_method`, the code raises `NameError`.

Vì lớp A chưa được định nghĩa khi Python biên dịch đoạn mã `factory_method` , nên đoạn mã này sẽ gây ra lỗi `NameError` .

The problem can be resolved by enclosing the return type `A` in quotes:

Vấn đề này có thể được giải quyết bằng cách đặt kiểu dữ liệu được trả về A trong dấu ngoặc kép.

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
class A:
    @classmethod
    def factory_method(cls) -> 'A':
        # ... method body goes here ...
```

A future version of Python may defer the evaluation of type annotations until runtime, making the enclosing quotes unnecessary (Python’s Steering Committee is evaluating various possibilities). You can preview this behavior using `from __future__ import annotations`.

Phiên bản tương lai của Python có thể trì hoãn việc kiểm tra các ghi chú về kiểu dữ liệu cho đến lúc chương trình được thực thi. Điều này sẽ làm cho việc sử dụng dấu ngoặc kép trở nên không cần thiết nữa (Ủy ban Quản lý Python đang xem xét nhiều khả năng khác nhau). Bạn có thể thử nghiệm tính năng này bằng cách sử dụng `from __future__ import annotations` .

The typing Module Mô-đun đánh chữ

The `typing` module supports type hints. It contains definitions that are useful when creating type annotations, including:

Mô-đun `typing` hỗ trợ việc sử dụng các gợi ý về kiểu dữ liệu. Nó chứa các định nghĩa hữu ích khi tạo các ghi chú về kiểu dữ liệu, bao gồm:

- Classes and functions for defining types

  Các lớp và hàm dùng để định nghĩa các kiểu dữ liệu
- Classes and functions for modifying type expressions

  Các lớp và hàm dùng để chỉnh sửa các biểu thức kiểu dữ liệu
- Abstract base classes (ABCs)

  Lớp cơ sở trừu tượng (Abstract Base Classes – ABCs)
- Protocols Quy trình/Thủ tục
- Utilities and decorators

  Các dịch vụ hỗ trợ và vật liệu trang trí
- Classes for defining custom types

  Các lớp dùng để định nghĩa các kiểu dữ liệu tùy chỉnh

Types Các loại

The initial implementations of the `typing` module included definitions of types corresponding to Python built-in containers and other types, as well as types from standard library modules. Many of these types have since been deprecated (see below), but some are still useful, since they do not correspond directly to any Python built-in type. **Table 5-1** lists the `typing` types still useful in Python 3.9 and later.

Các triển khai ban đầu của mô-đun `typing` bao gồm định nghĩa các kiểu tương ứng với các container tích hợp sẵn trong Python và các kiểu khác, cũng như các kiểu từ các mô-đun thư viện tiêu chuẩn. Nhiều kiểu này đã bị loại bỏ (xem bên dưới), nhưng một số vẫn hữu ích, vì chúng không tương ứng trực tiếp với bất kỳ kiểu tích hợp sẵn nào của Python. Bảng 5-1 liệt kê các kiểu `typing` vẫn còn hữu ích trong Python 3.9 trở lên.

Table 5-1. Useful definitions in the `typing` module

Bảng 5-1. Các định nghĩa hữu ích trong mô-đun `typing`

Type Loại	**Description** Mô tả
`Any`	Matches any type. Phù hợp với mọi loại.
`AnyStr`	Equivalent to `str | bytes`. `AnyStr` is meant to be used to annotate function arguments and return types where either string type is acceptable, but the types should not be mixed between multiple arguments, or arguments and return types. Tương đương với `str | bytes` . `AnyStr` được dùng để ghi chú các tham số của hàm và kiểu dữ liệu được trả về, trong trường hợp kiểu dữ liệu là chuỗi ký tự. Tuy nhiên, các kiểu dữ liệu này không được phép xuất hiện trong cùng một tham số, hoặc giữa các tham số và kiểu dữ liệu được trả về.
`BinaryIO`	Matches streams with binary (`bytes`) content such as those returned from `open` with `mode='b'`, or `io.BytesIO`. Các luồng dữ liệu này tương ứng với nội dung dạng nhị phân (`bytes`), chẳng hạn như những dữ liệu được trả về từ open cùng với `mode='b'` hoặc `io.BytesIO` .
`Callable`	`Callable[[`*`argument_type`*`, ...], `*`return_type`*`]` Defines the type signature for a callable object. Takes a list of types corresponding to the arguments to the callable, and a type for the return value of the function. If the callable takes no arguments, indicate this with an empty list, `[]`. If the callable has no return value, use **None** for *`return_type`*. Xác định kiểu dữ liệu của đối tượng có thể được gọi là hàm. Đối số là danh sách các kiểu dữ liệu tương ứng với các đối số được truyền vào hàm, cùng với kiểu dữ liệu của giá trị trả về bởi hàm. Nếu hàm không nhận đối số nào, hãy sử dụng danh sách rỗng `[]` . Nếu hàm không trả về giá trị nào, hãy sử dụng **None** thay cho *`return_type`* .

`IO`	Equivalent to `BinaryIO` | `TextIO`. Tương đương với `BinaryIO` | `TextIO` .
`Literal` `[expression,...]`	**3.8+** Specifies a list of valid values that the variable may take. 3.8+ Xác định danh sách các giá trị hợp lệ mà biến có thể nhận được.
`LiteralString`	**3.11+** Specifies a `str` that must be implemented as a literal quoted value. Used to guard against leaving code open to injection attacks. 3.11+ Xác định một giá trị `str` cần được biểu diễn dưới dạng chuỗi ký tự được đặt trong dấu ngoặc. Cách làm này nhằm ngăn chặn các cuộc tấn công kiểu "injection".
`NoReturn`	Use as the return type for functions that "run forever," such as those that call `http.serve_forever` or `event_loop.run_forever` without returning. This is *not* intended for functions that simply return with no explicit value; for those use `->` **None**. More discussion of return types can be found in "Adding Type Annotations to Existing Code (Gradual Typing)". Hãy sử dụng kiểu dữ liệu này làm kiểu dữ liệu trả về cho các hàm "chạy mãi không dừng", tức là những hàm gọi các hàm `http.serve_forever` hoặc `event_loop.run_forever` mà không trả về giá trị nào. Kiểu dữ liệu này không phù hợp để sử dụng cho các hàm chỉ đơn giản trả về giá trị "không xác định"; đối với những trường hợp như vậy, hãy sử dụng các kiểu dữ liệu `->` và **None** . Thông tin chi tiết về các kiểu dữ liệu trả về có thể được tìm thấy trong phần "Thêm ghi chú về kiểu dữ liệu vào mã nguồn hiện có (Typing Gradual)".
`Self`	**3.11+** Use as the return type for instance functions that **`return`** `self` (and in a few other cases, as exemplified in PEP 673). 3.11+ Có thể được sử dụng làm kiểu dữ liệu trả về cho các hàm instance, như đã được đề cập trong **`return`** `self` (và trong một vài trường hợp khác, như được minh họa trong PEP 673).
`TextIO`	Matches streams with text (`str`) content, such as those returned from `open` with `mode='t'`, or `io.StringIO`. Tìm các luồng dữ liệu tương ứng với nội dung văn bản (`str`), chẳng hạn như những dữ liệu được trả về từ open cùng với `mode='t'` hoặc `io.StringIO` .

-3.9 Prior to 3.9, the definitions in the `typing` module were used to create types representing built-in types, such as `List[int]` for a list of `int`s. From 3.9 onward, these names are deprecated, as their corresponding built-in or standard library types now support the `[]` syntax: a list of `int`s is now simply typed using `list[int]`. Table 5-2 lists the definitions from the `typing` module that were necessary prior to Python 3.9 for type annotations using built-in types.

Trước phiên bản 3.9, các định nghĩa trong mô-đun `typing` được sử dụng để tạo ra các kiểu dữ liệu tương ứng với các kiểu dữ liệu có sẵn trong ngôn ngữ lập trình. Ví dụ, `List[int]` được dùng để biểu diễn một danh sách các phần tử thuộc kiểu `int` . Từ phiên bản 3.9 trở đi, các tên này không còn được sử dụng nữa, vì các kiểu dữ liệu tương ứng trong thư viện tiêu chuẩn hiện đã hỗ trợ cú pháp `[]` . Việc biểu diễn một danh sách các phần tử thuộc kiểu `int` giờ đây chỉ cần sử dụng ký hiệu `list[int]` . Bảng 5-2 liệt kê các định nghĩa trong mô-đun `typing` mà cần thiết để sử dụng các kiểu dữ liệu có sẵn trong Python 3.9.

Table 5-2. Python built-in types and their pre-3.9 definitions in the `typing` module

Bảng 5-2. Các kiểu dữ liệu có sẵn trong Python và cách định nghĩa của chúng trước phiên bản 3.9 trong mô-đun `typing` .

Built-in type Loại được tích hợp sẵn	**Pre-3.9 typing module equivalent** **Tương đương với mô-đun nhập liệu trước phiên bản 3.9**
`dict`	`Dict`
`frozenset`	`FrozenSet`
`list`	`List`
`set`	`Set`
`str`	`Text`
`tuple`	`Tuple`

`type`	`Type`
`collections.ChainMap`	`ChainMap`
`collections.Counter`	`Counter`
`collections.defaultdict`	`DefaultDict`
`collections.deque`	`Deque`
`collections.OrderedDict`	`OrderedDict`
`re.Match`	`Match`
`re.Pattern`	`Pattern`

Type Expression Parameters

Các tham số của biểu thức kiểu dữ liệu này

Some types defined in the `typing` module modify other type expressions. The types listed in **Table 5-3** provide additional typing information or constraints for the modified types in *type_expression*.

Một số kiểu dữ liệu được định nghĩa trong mô-đun `typing` có khả năng thay đổi cách biểu diễn của các kiểu dữ liệu khác. Các kiểu dữ liệu được liệt kê trong Bảng 5-3 cung cấp thêm thông tin hoặc ràng buộc liên quan đến các kiểu dữ liệu bị thay đổi trong mô-đun *type_expression* .

Table 5-3. Type expression parameters
Bảng 5-3. Các tham số biểu thức kiểu dữ liệu

Parameter **Tham số**	**Usage and description** **Cách sử dụng và mô tả**
`Annotated`	`Annotated[`*type_expression, expression, ...*`]` **3.9+** Extends the *type_expression* with additional metadata. The extra metadata values for function *fn* can be retrieved at runtime using `get_type_hints(`*fn*`, include_extras=True)`. 3.9+ Giúp mở rộng thông tin về *type_expression* bằng cách thêm các siêu dữ liệu vào. Các giá trị siêu dữ liệu liên quan đến hàm *fn* có thể được truy xuất trong quá trình thực thi chương trình, bằng cách sử dụng `get_type_hints(`*fn*`, include_extras=True)` .
`ClassVar`	`ClassVar[`*type_expression*`]` Indicates that the variable is a class variable, and should not be assigned as an instance variable. Điều này cho thấy rằng biến đó là biến thuộc lớp, và không nên được gán giá trị như một biến thuộc instance.
`Final`	`Final[`*type_expression*`]` **3.8+** Indicates that the variable should not be written to or overridden in a subclass. 3.8+ có nghĩa là biến này không nên được ghi đè hoặc thay đổi giá trị trong các lớp con.
`Optional`	`Optional[`*type_expression*`]` Equivalent to *type_expression* | **None**. Often used for named arguments with a default value of **None**. (`Optional` does not automatically define **None** as the default value, so you must still follow it with =**None** in a function signature.) **3.10+** With the availability of the | operator for specifying alternative type attributes, there is a growing consensus to prefer *type_expression* | **None** over using `Optional[`*type_expression*`]`. `Optional[`*type_expression*`]` Tương đương với *type_expression* | **None** . Thường được sử dụng cho các đối số có tên, trong đó giá trị mặc định là **None** . (`Optional` không tự động đặt **None** làm giá trị mặc định; vì vậy bạn vẫn cần phải chỉ định giá trị mặc định bằng = **None** trong thông số hàm.) Từ phiên bản 3.10 trở đi, với sự xuất hiện của toán tử | dùng để chỉ định các thuộc tính kiểu dữ liệu khác nhau, người ta ngày càng ưa chuộng việc sử dụng *type_expression* | **None** thay vì `Optional[`*type_expression*`]` .

Abstract Base Classes Lớp cơ sở trừu tượng

Just as for built-in types, the initial implementations of the `typing` module included definitions of types corresponding to abstract base classes in the `collections.abc` module. Many of these types have since been deprecated (see below), but two definitions have been retained as aliases to ABCs in `collections.abc` (see Table 5-4).

Cũng giống như các kiểu dữ liệu có sẵn, các phiên bản đầu tiên của mô-đun `typing` cũng bao gồm các định nghĩa về các kiểu dữ liệu tương ứng với các lớp cơ sở trừu tượng trong mô-đun `collections.abc` . Nhiều kiểu dữ liệu này sau đó đã bị loại bỏ (xem phần dưới), nhưng hai kiểu dữ liệu vẫn được giữ lại, dùng như tên viết tắt cho các lớp cơ sở trừu tượng trong mô-đun `collections.abc` (xem Bảng 5-4).

Table 5-4. Abstract base class aliases
Bảng 5-4. Tên gọi khác của lớp cơ sở trừu tượng

Type Loại	**Method subclasses must implement** **Các lớp con của lớp Method phải triển khai các phương thức được yêu cầu.**
`Hashable`	`__hash__`
`Sized`	`__len__`

-3.9 Prior to Python 3.9, the following definitions in the `typing` module represented abstract base classes defined in the `collections.abc` module, such as `Sequence[int]` for a sequence of `int`s. From 3.9 onward, these names in the `typing` module are deprecated, as their corresponding types in `collections.abc` now support the `[]` syntax:

Trước phiên bản Python 3.9, các định nghĩa trong mô-đun `typing` thực chất là các lớp cơ sở trừu tượng được định nghĩa trong mô-đun `collections.abc` . Ví dụ, `Sequence[int]` chính là lớp cơ sở dùng để biểu diễn một dãy các đối tượng thuộc kiểu `int` . Từ phiên bản 3.9 trở đi, các tên này trong mô-đun `typing` đã bị loại bỏ, vì các kiểu dữ liệu tương ứng trong mô-đun `collections.abc` hiện đã hỗ trợ cú pháp `[]` .

- `AbstractSet`
- `AsyncContextManager`
- `AsyncGenerator`
- `AsyncIterable`
- `AsyncIterator`
- `Awaitable`
- `ByteString`
- `Collection`
- `Container`
- `ContextManager`
- `Coroutine`
- `Generator`
- `ItemsView`
- `Iterable`
- `Iterator`
- `KeysView`
- `Mapping`
- `MappingView`
- `MutableMapping`
- `MutableSequence`
- `MutableSet`
- `Reversible`
- `Sequence`
- `ValuesView`

Protocols Quy trình/Thủ tục

The `typing` module defines several *protocols*, which are similar to what some other languages call "interfaces." Protocols are abstract base classes intended to concisely express constraints on a type, ensuring it contains certain methods. Each protocol currently defined in the `typing` module relates to a single special method, and its name starts with `Supports` followed by the name of the method (however, other libraries, such as those defined in `typeshed`, need not follow the same constraints). Protocols can be used as minimal abstract classes to determine a class's support for that protocol's capabilities: all that a class needs to do to comply with a protocol is to implement the protocol's special method(s).

Mô-đun `typing` định nghĩa nhiều giao thức khác nhau. Những giao thức này tương tự như những gì một số ngôn ngữ lập trình khác gọi là "giao diện". Các giao thức là những lớp cơ sở trừu tượng, có chức năng mô tả các yêu cầu cần được thỏa mãn bởi một lớp cụ thể, đồng thời đảm bảo rằng lớp đó phải chứa các phương thức cần thiết. Hiện tại, mỗi giao thức được định nghĩa trong mô-đun `typing` đều liên quan đến một phương thức cụ thể. Tên của giao thức bắt đầu bằng ký tự `Supports` , sau đó là tên của phương thức đó. Tuy nhiên, các thư viện khác, chẳng hạn như những thư viện được định nghĩa trong mô-đun `typeshed` , không nhất thiết phải tuân theo quy tắc này. Các giao thức có thể được sử dụng như những lớp cơ sở trừu tượng để xác định liệu một lớp có hỗ trợ các tính năng của giao thức đó hay không. Điều mà một lớp cần làm để tuân thủ một giao thức chính là triển khai các phương thức được định nghĩa trong giao thức đó.

Table 5-5 lists the protocols defined in the `typing` module.

Bảng 5-5 liệt kê các giao thức được định nghĩa trong mô-đun `typing` .

Table 5-5. Protocols in the `typing` module and their required methods

Bảng 5-5. Các giao thức trong mô-đun `typing` và các phương thức cần thiết của chúng

Protocol Quy trình/Thủ tục	**Has method** Có phương pháp.
`SupportsAbs`	`__abs__`
`SupportsBytes`	`__bytes__`
`SupportsComplex`	`__complex__`
`SupportsFloat`	`__float__`
`SupportsIndex` **3.8+**	`__index__`
`SupportsInt`	`__int__`
`SupportsRound`	`__round__`

A class does not have to explicitly inherit from a protocol in order to satisfy `issubclass(`*`cls`*`,` *`protocol_type`*`)`, or for its instances to satisfy `isinstance(`*`obj`*`,` *`protocol_type`*`)`. The class simply has to implement the method(s) defined in the protocol. Imagine, for example, a class implementing Roman numerals:

Một lớp không nhất thiết phải kế thừa trực tiếp từ một giao thức nào đó để có thể đáp ứng các yêu cầu `issubclass(`*`cls`*`,` *`protocol_type`* `)` , hoặc để các đối tượng thuộc lớp đó đáp ứng các yêu cầu `isinstance(`*`obj`*`,` *`protocol_type`* `)` . Lớp đó chỉ cần triển khai các phương thức được định nghĩa trong giao thức đó. Ví dụ, hãy tưởng tượng một lớp dùng để biểu diễn các con số La Mã:

Explain this code Hãy giải thích đoạn mã này. | **Translate into** Dịch sang tiếng Việt ▾

```
class RomanNumeral:
    """Class representing some Roman numerals and their int
       values.
    """
    int_values = {'I': 1, 'II': 2, 'III': 3, 'IV': 4, 'V': 5}

    def __init__(self, label: str):
        self.label = label

    def __int__(self) -> int:
        return RomanNumeral.int_values[self.label]
```

To create an instance of this class (to, say, represent a sequel in a movie title) and get its value, you could use the following code:

Để tạo một đối tượng thuộc lớp này (ví dụ: để biểu diễn phần tiếp theo của một bộ phim), và lấy giá trị của đối tượng đó, bạn có thể sử dụng đoạn mã sau:

Explain this code Hãy giải thích đoạn mã này. | **Translate into** Dịch sang tiếng Việt ▾

```
>>> movie_sequel = RomanNumeral('II')
>>> print(int(movie_sequel))
```

Explain this code Hãy giải thích đoạn mã này. | **Translate into** Dịch sang tiếng Việt ▾

```
2
```

`RomanNumeral` satisfies `issubclass`, and `isinstance` checks with `SupportsInt` because it implements `__int__`, even though it does not inherit explicitly from the protocol class `SupportsInt`:[4]

`RomanNumeral` thỏa mãn các điều kiện của `issubclass` . Ngoài ra, `isinstance` cũng tương thích với `SupportsInt` , vì `isinstance` triển khai các chức năng được định nghĩa trong `__int__` . Mặc dù vậy, `isinstance` không kế thừa trực tiếp từ lớp giao thức `SupportsInt` . [4]

Explain this code Hãy giải thích đoạn mã này. | **Translate into** Dịch sang tiếng Việt ▾

```
>>> issubclass(RomanNumeral, typing.SupportsInt)
```

Explain this code Hãy giải thích đoạn mã này. | **Translate into** Dịch sang tiếng Việt ▾

```
True
```

Explain this code Hãy giải thích đoạn mã này. Translate into Dịch sang tiếng Việt▾

```
>>> isinstance(movie_sequel, typing.SupportsInt)
```

Explain this code Hãy giải thích đoạn mã này. Translate into Dịch sang tiếng Việt▾

```
True
```

Utilities and Decorators

Các dịch vụ hỗ trợ và vật liệu trang trí

Table 5-6 lists commonly used functions and decorators defined in the `typing` module; it's followed by a few examples.

Bảng 5-6 liệt kê các hàm và tiện ích thường được sử dụng, được định nghĩa trong mô-đun `typing` . Sau đó là một vài ví dụ minh họa.

Table 5-6. Commonly used functions and decorators defined in the `typing` module

Bảng 5-6. Các hàm và tiện ích thường được sử dụng, được định nghĩa trong mô-đun `typing` .

Function/decorator Hàm/funktor trang trí	**Usage and description** Cách sử dụng và mô tả
`cast`	`cast(`*`type`*`,` *`var`*`)` Signals to the static type checker that *var* should be considered as type *type*. Returns *var*; at runtime there is no change, conversion, or validation of *var*. See the example after the table. Đây là tín hiệu gửi đến công cụ kiểm tra kiểu dữ liệu, báo rằng *var* nên được coi là kiểu dữ liệu *type* . Kết quả trả về là *var* . Trong quá trình thực thi chương trình, giá trị của *var* không bị thay đổi, chuyển đổi hay xác thực gì cả. Xem ví dụ minh họa ở phía sau bảng.
`final`	`@final` **3.8+** Used to decorate a method in a class definition, to warn if the method is overridden in a subclass. Can also be used as a class decorator, to warn if the class itself is being subclassed. 3.8+ Thường được sử dụng để ghi chú về một phương thức trong định nghĩa lớp, nhằm cảnh báo khi phương thức đó bị thay đổi trong lớp con. Nó cũng có thể được dùng như một công cụ ghi chú cho chính lớp đó, nhằm cảnh báo khi lớp đó bị kế thừa bởi lớp con nào đó.
`get_args`	`get_args(`*`custom_type`*`)` Returns the arguments used to construct a custom type. Trả về các đối số được sử dụng để tạo ra một kiểu dữ liệu tùy chỉnh.
`get_origin`	`get_origin(`*`custom_type`*`)` **3.8+** Returns the base type used to construct a custom type. 3.8+ Trả về kiểu dữ liệu cơ bản được sử dụng để tạo ra kiểu dữ liệu tùy chỉnh.
`get_type_hints`	`get_type_hints(`*`obj`*`)` Returns results as if accessing *`obj`*`.__annotations__`. Can be called with optional `globalns` and `localns` namespace arguments to resolve forward type references given as strings, and/or with optional Boolean `include_extras` argument to include any nontyping annotations added using `Annotations`. Trả về kết quả tương tự như khi truy cập vào *`obj`* `.__annotations__` . Có thể gọi hàm này với các tham số tên không gian `globalns` và `localns` để xử lý các tham chiếu kiểu dữ liệu được biểu diễn dưới dạng chuỗi ký tự. Ngoài ra, cũng có thể sử dụng tham số Boolean `include_extras` để bao gồm các ghi chú liên quan đến kiểu dữ liệu được thêm vào bằng cách sử dụng `Annotations` .
`NewType`	`NewType(`*`type_name`*`,` *`type`*`)` Defines a custom type derived from *type*. *type_name* is a string that should match the local variable to which the `NewType` is being assigned. Useful for distinguishing different uses for common types, such as a `str` used for an

	employee name versus a `str` used for a department name. See “NewType” for more on this function. Định nghĩa một kiểu dữ liệu tùy chỉnh được kế thừa từ *`type`* . *`type_name`* là chuỗi ký tự cần phải trùng khớp với tên biến cục bộ mà giá trị NewType sẽ được gán vào. Điều này rất hữu ích để phân biệt các cách sử dụng khác nhau của các kiểu dữ liệu thông thường; ví dụ: `str` có thể được dùng để lưu tên nhân viên, trong khi `str` được dùng để lưu tên bộ phận. Xem mục “NewType” để biết thêm thông tin về hàm này.
`no_type_check`	`@no_type_check` Used to indicate that annotations are not intended to be used as type information. Can be applied to a class or function. Dùng để chỉ rằng các ghi chú này không nhằm mục đích được sử dụng như thông tin về kiểu dữ liệu. Có thể được áp dụng cho cả lớp lẫn hàm.
`no_type_check_decorator`	`@no_type_check_decorator` Used to add `no_type_check` behavior to another decorator. Dùng để thêm hành vi `no_type_check` vào một decorator khác.
`overload`	`@overload` Used to allow defining multiple methods with the same name but differing types in their signatures. See the example after the table. Trước đây, cách này được sử dụng để cho phép định nghĩa nhiều phương thức có cùng tên, nhưng khác nhau về kiểu dữ liệu được truyền vào. Xem ví dụ ở phía dưới bảng.
`runtime_checkable`	`@runtime_checkable` **3.8+** Used to add `isinstance` and `issubclass` support for custom protocol classes. See “Using Type Annotations at Runtime” for more on this decorator. 3.8+ được sử dụng để hỗ trợ việc sử dụng các lớp giao thức tùy chỉnh, cụ thể là các lớp sử dụng các ký hiệu `isinstance` và `issubclass` . Bạn có thể tìm hiểu thêm về tính năng này trong phần “Sử dụng các ghi chú về kiểu dữ liệu trong quá trình thực thi chương trình”.
`TypeAlias`	*`name`*`: TypeAlias =` *`type_expression`* **3.10+** Used to distinguish the definition of a type alias from a simple assignment. Most useful in cases where *`type_expression`* is a simple class name or a string value referring to a class that is not yet defined, which might look like an assignment. `TypeAlias` may only be used at module scope. A common use is to make it easier to consistently reuse a lengthy type expression, e.g.: `Number: TypeAlias = int | float | Fraction`. See “TypeAlias” for more on this annotation. 3.10+ Được sử dụng để phân biệt giữa khái niệm “biệt danh kiểu dữ liệu” và thao tác gán giá trị đơn giản. Điều này rất hữu ích trong những trường hợp mà *`type_expression`* chỉ là tên của một lớp hoặc giá trị chuỗi tham chiếu đến một lớp chưa được định nghĩa trước đó; những trường hợp này có vẻ giống như thao tác gán giá trị thông thường. `TypeAlias` chỉ có thể được sử dụng trong phạm vi của một mô-đun. Một ví dụ phổ biến về cách sử dụng `TypeAlias` là để tạo điều kiện thuận lợi cho việc tái sử dụng các biểu thức kiểu dữ liệu dài một cách dễ dàng, ví dụ: `Number: TypeAlias = int | float | Fraction` . Xem thêm phần “TypeAlias” để biết thêm thông tin về kiểu ghi chú này.
`type_check_only`	`@type_check_only` Used to indicate that the class or function is only used at type-checking time and is not available at runtime. Dùng để chỉ rằng lớp hoặc hàm đó chỉ được sử dụng trong quá trình kiểm tra kiểu dữ liệu, và không được sử dụng trong quá trình thực thi chương trình.
`TYPE_CHECKING`	A special constant that static type checkers evaluate as **`True`** but that is set to **`False`** at runtime. Use this to skip imports of large, slow-to-import modules used solely to support type checking (so that the import is not needed at runtime). Đó là một hằng số đặc biệt: các công cụ kiểm tra kiểu dữ liệu sẽ xử lý giá trị của hằng số

	này như **`True`** , nhưng trong quá trình thực thi chương trình, giá trị của nó sẽ được đặt thành **`False`** . Hãy sử dụng hằng số này để tránh việc nhập các mô-đun lớn, mất nhiều thời gian để nhập vào chương trình; những mô-đun này chỉ được sử dụng nhằm hỗ trợ công việc kiểm tra kiểu dữ liệu mà thôi. Nhờ vậy, việc nhập các mô-đun này sẽ không còn cần thiết trong quá trình thực thi chương trình.
`TypeVar`	`TypeVar(`*`type_name`*`, *`*`types`*`)` Defines a type expression element for use in complex generic types using `Generic`. *`type_name`* is a string that should match the local variable to which the `TypeVar` is being assigned. If *`types`* are not given, then the associated `Generic` will accept any type. If *`types`* are given, then the `Generic` will only accept instances of any of the provided types or their subclasses. Also accepts the named Boolean arguments `covariant` and `contravariant` (both defaulting to `False`), and the argument `bound`. These are described in more detail in **"Generics and TypeVars"** and in the **typing module docs**. Định nghĩa một phần tử biểu diễn kiểu để sử dụng trong các kiểu generic phức tạp, sử dụng `Generic` . *`type_name`* là một chuỗi, và nó phải khớp với biến cục bộ mà `TypeVar` đang được gán. Nếu *`types`* không được cung cấp, thì `Generic` sẽ chấp nhận bất kỳ kiểu nào. Nếu *`types`* được cung cấp, thì `Generic` sẽ chỉ chấp nhận các thực thể của bất kỳ kiểu nào được cung cấp hoặc các lớp con của chúng. Ngoài ra, nó cũng chấp nhận các đối số Boolean có tên `covariant` và `contravariant` (cả hai mặc định là `False`), và đối số `bound` . Các thông tin này được mô tả chi tiết hơn trong phần "Generics và TypeVars" và trong tài liệu của mô-đun typing.

Use `overload` at type-checking time to flag named arguments that must be used in particular combinations. In this case, `fn` must be called with either a `str` key and `int` value pair, or with a single `bool` value:

Hãy sử dụng `overload` trong quá trình kiểm tra kiểu dữ liệu, để đánh dấu các đối số có tên mà bắt buộc phải được sử dụng theo những cách nhất định. Trong trường hợp này, `fn` phải được gọi cùng với cặp giá trị `str` và `int` , hoặc cùng với giá trị `bool` duy nhất.

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt ▾

```
@typing.overload
def fn(*, key: str, value: int):
    ...

@typing.overload
def fn(*, strict: bool):
    ...

def fn(**kwargs):
    # implementation goes here, including handling of differing
    # named arguments
    pass

# valid calls
fn(key='abc', value=100)
fn(strict=True)

# invalid calls
fn(1)
fn('abc')
fn('abc', 100)
fn(key='abc')
fn(True)
fn(strict=True, value=100)
```

Note that the `overload` decorator is used purely for static type checking. To actually dispatch to different methods based on a parameter type at runtime, use `functools.singledispatch`.

Lưu ý rằng decorator `overload` chỉ được sử dụng nhằm mục đích kiểm tra kiểu dữ liệu tĩnh. Để có thể gọi các phương thức khác nhau tùy theo kiểu dữ liệu của tham số tại thời điểm thực thi chương trình, hãy sử dụng decorator `functools.singledispatch` .

Use the `cast` function to force a type checker to treat a variable as being of a particular type, within the scope of the cast:

Hãy sử dụng hàm `cast` để buộc bộ kiểm tra kiểu dữ liệu phải coi một biến thuộc một kiểu dữ liệu cụ thể nào đó, trong phạm vi của lệnh chuyển đổi kiểu dữ liệu đó.

Explain this code Hãy giải thích đoạn mã này.

Translate into Dịch sang tiếng Việt▾

```
def func(x: list[int] | list[str]):
    try:
        return sum(x)
    except TypeError:
        x = cast(list[str], x)
        return ','.join(x)
```

USE CAST WITH CAUTION

HÃY SỬ DỤNG CHỨC NĂNG “CAST” MỘT CÁCH THẬN TRỌNG.

`cast` is a way of overriding any inferences or prior annotations that may be present at a particular place in your code. It may hide actual type errors in your code, rendering the type-checking pass incomplete or inaccurate. The `func` in the preceding example raises no `mypy` warnings itself, but fails at runtime if passed a list of mixed `int`s and `str`s.

`cast` là cách để “ghi đè” bất kỳ thông tin nào liên quan đến kiểu dữ liệu được ghi chú trước đó trong mã nguồn. Việc này có thể che giấu các lỗi liên quan đến kiểu dữ liệu trong mã nguồn, khiến quá trình kiểm tra kiểu dữ liệu không được thực hiện đầy đủ hoặc chính xác. Trong ví dụ trên, `func` không gây ra bất kỳ cảnh báo nào; tuy nhiên, nó sẽ gây lỗi khi chạy chương trình nếu người dùng cung cấp một danh sách chứa cả `int` và `str` .

Defining Custom Types Định nghĩa các kiểu dữ liệu tùy chỉnh

Just as Python’s **`class`** syntax permits the creation of new runtime types and behavior, the `typing` module constructs discussed in this section enable the creation of specialized type expressions for advanced type checking.

Cũng giống như cú pháp **`class`** trong Python, cho phép tạo ra các kiểu dữ liệu và chức năng mới trong quá trình thực thi chương trình, các mô-đun `typing` được đề cập trong phần này cũng giúp tạo ra các biểu thức kiểu dữ liệu chuyên dụng, nhằm hỗ trợ việc kiểm tra kiểu dữ liệu một cách hiệu quả hơn.

The `typing` module includes three classes from which your classes can inherit to get type definitions and other default features, listed in Table 5-7.

Mô-đun `typing` bao gồm ba lớp mà các lớp của bạn có thể kế thừa từ đó để nhận được các định nghĩa về kiểu dữ liệu và các tính năng mặc định khác. Danh sách các lớp này được trình bày trong Bảng 5-7.

Table 5-7. Base classes for defining custom types

Bảng 5-7. Các lớp cơ sở dùng để định nghĩa các kiểu dữ liệu tùy chỉnh

`Generic`	`Generic[`*`type_var`*`, ...]` Defines a type-checking abstract base class for a class whose methods reference one or more `TypeVar`-defined types. Generics are described in more detail in the following subsection. Đây là một lớp cơ sở trừu tượng dùng để kiểm tra kiểu dữ liệu của các lớp mà các phương thức của chúng tham chiếu đến một hoặc nhiều kiểu dữ liệu được định nghĩa bởi `TypeVar` . Các khái niệm liên quan đến việc sử dụng các kiểu dữ liệu tổng quát sẽ được trình bày chi tiết hơn ở phần tiếp theo.
`NamedTuple`	`NamedTuple` A typed implementation of `collections.namedtuple`. See “NamedTuple” for further details and examples. Đây là cách triển khai `collections.namedtuple` dưới dạng văn bản. Để biết thêm chi tiết và ví dụ minh họa, xin xem mục “NamedTuple”.
`TypedDict`	`TypedDict` **3.8+** Defines a type-checking `dict` that has specific keys and value types for each key. See “TypedDict” for details. 3.8+ Định nghĩa một cấu trúc dữ liệu có khả năng kiểm tra kiểu dữ liệu: mỗi khóa trong cấu trúc này đều tương ứng với một kiểu giá trị cụ thể. Xem thêm phần “TypedDict” để biết chi tiết.

Generics and TypeVars Các kiểu dữ liệu tổng quát và TypeVars

Generics are types that define a template for classes that can adapt the type annotations of their method signatures based on one or more type parameters. For instance, `dict` is a generic that takes two type parameters: the type for the dictionary keys and the type for the dictionary values. Here is how `dict` might be used to define a dictionary that maps color names to RGB triples:

Các kiểu dữ liệu tổng quát là những kiểu dữ liệu giúp xác định “khuôn mẫu” cho các lớp khác nhau. Nhờ đó, các phương thức thuộc các lớp này có thể sử dụng các tham số kiểu dữ liệu khác nhau tùy theo nhu cầu. Ví dụ, `dict` là một kiểu dữ liệu tổng quát, nhận hai tham số kiểu dữ liệu: kiểu dữ liệu của các khóa trong từ điển và kiểu dữ liệu của các giá trị trong từ điển. Dưới đây là cách sử dụng `dict` để định nghĩa một từ điển biến tên màu sắc thành các giá trị RGB tương ứng:

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
color_lookup: dict[str, tuple[int, int, int]] = {}
```

The variable `color_lookup` will support statements like:

Biến `color_lookup` sẽ hỗ trợ các câu lệnh như sau:

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
color_lookup['red'] = (255, 0, 0)
color_lookup['red'][2]
```

However, the following statements generate `mypy` errors, due to a mismatched key or value type:

Tuy nhiên, các câu lệnh sau gây ra lỗi `mypy` , do sự không tương ứng giữa kiểu dữ liệu của khóa và giá trị:

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
color_lookup[0]
```

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
error: Invalid index type "int" for "dict[str, tuple[int, int, int]]";
expected type "str"
```

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
color_lookup['red'] = (255, 0, 0, 0)
```

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
error: Incompatible types in assignment (expression has type
"tuple[int, int, int, int]", target has type "tuple[int, int, int]")
```

Generic typing permits the definition of behavior in a class that is independent of the specific types of the objects that class works with. Generics are often used for defining container types, such as `dict`, `list`, `set`, etc. By defining a generic type, we avoid the necessity of exhaustively defining types for `DictOfStrInt`, `DictOfIntEmployee`, and so on. Instead, a generic `dict` is defined as `dict[KT, VT]`, where *KT* and *VT* are placeholders for the `dict`'s key type and value type, and the specific types for any particular `dict` can be defined when the `dict` is instantiated.

Lập trình hướng đối tượng cho phép định nghĩa các hành vi của một lớp một cách độc lập với các kiểu dữ liệu cụ thể mà lớp đó sẽ làm việc với. Các kỹ thuật lập trình hướng đối tượng thường được sử dụng để định nghĩa các kiểu dữ liệu dùng để lưu trữ dữ liệu, chẳng hạn như `dict` , `list` , `set` ... Khi định nghĩa một kiểu dữ liệu tổng quát, chúng ta không cần phải định nghĩa riêng từng kiểu dữ liệu cụ thể như `DictOfStrInt` , `DictOfIntEmployee` ... Thay vào đó, kiểu dữ liệu tổng quát `dict` được định nghĩa dưới dạng `dict[KT, VT ]` . Trong đó, *KT* và *VT* là các vị trí dành để chứa kiểu dữ liệu khóa và kiểu dữ liệu giá trị của đối tượng thuộc kiểu `dict`. Các kiểu dữ liệu cụ thể của từng đối tượng thuộc kiểu `dict` sẽ được xác định khi đối tượng đó được tạo ra.

As an example, let's define a hypothetical generic class: an accumulator that can be updated with values, but which also supports an `undo` method. Since the accumulator is a generic container, we declare a `TypeVar` to represent the type of the contained objects:

Ví dụ, hãy định nghĩa một lớp tổng quát giả định: đó là một cấu trúc dữ liệu có thể được cập nhật bằng các giá trị nhất định, đồng thời hỗ trợ phương thức `undo` . Vì đây là một cấu trúc dữ liệu tổng quát, chúng ta cần khai báo một tham số `TypeVar` để xác định loại dữ liệu được lưu trữ bên trong.

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
import typing
T = typing.TypeVar('T')
```

The `Accumulator` class is defined as a subclass of `Generic`, with `T` as a type parameter. Here is the class declaration and its `__init__` method, which creates a contained list, initially empty, of objects of type `T`:

Lớp `Accumulator` được định nghĩa là lớp con của lớp `Generic` , với `T` là tham số kiểu dữ liệu. Dưới đây là thông tin về lớp này cùng phương thức `__init__` của nó: Phương thức này tạo ra một danh sách chứa các đối tượng thuộc kiểu `T` , ban đầu danh sách này trống.

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
class Accumulator(typing.Generic[T]):
    def __init__(self):
        self._contents: list[T] = []
```

To add the `update` and `undo` methods, we define arguments that reference the contained objects as being of type `T`:

Để thêm các phương thức `update` và `undo` , chúng ta cần định nghĩa các tham số sao cho chúng trỏ đến các đối tượng nằm bên trong, với kiểu dữ liệu là `T` .

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
    def update(self, *args: T) -> None:
        self._contents.extend(args)

    def undo(self) -> None:
        # remove last value added
        if self._contents:
            self._contents.pop()
```

Lastly, we add `__len__` and `__iter__` methods so that `Accumulator` instances can be iterated over:

Cuối cùng, chúng ta thêm các phương thức `__len__` và `__iter__` để có thể lặp qua các đối tượng thuộc loại `Accumulator` .

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
    def __len__(self) -> int:
        return len(self._contents)

    def __iter__(self) -> typing.Iterator[T]:
        return iter(self._contents)
```

Now this class can be used to write code using `Accumulator[int]` to collect a number of `int` values:

Bây giờ, lớp này có thể được sử dụng để viết mã mà trong đó, ký hiệu `Accumulator[int]` được dùng để thu thập một số giá trị thuộc loại `int` .

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
acc: Accumulator[int] = Accumulator()
acc.update(1, 2, 3)
print(sum(acc))  # prints 6
acc.undo()
print(sum(acc))  # prints 3
```

Because `acc` is an `Accumulator` containing `int`s, the following statements generate `mypy` error messages:

Vì `acc` là một `Accumulator` chứa các `int` , nên các câu lệnh sau sẽ tạo ra các thông báo lỗi loại `mypy` .

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
acc.update('A')
```

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
error: Argument 1 to "update" of "Accumulator" has incompatible type
"str"; expected "int"
```

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
print(''.join(acc))
```

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
error: Argument 1 to "join" of "str" has incompatible type
"Accumulator[int]"; expected "Iterable[str]"
```

Restricting TypeVar to specific types

Hạn chế việc sử dụng TypeVar chỉ cho các kiểu dữ liệu cụ thể.

Nowhere in our `Accumulator` class do we ever invoke methods directly on the contained `T` objects themselves. For this example, the `T` `TypeVar` is purely untyped, so type checkers like `mypy` cannot infer the presence of any attributes or methods of the `T` objects. If the generic needs to access attributes of the `T` objects it contains, then `T` should be defined using a modified form of `TypeVar`.

Trong lớp `Accumulator` của chúng ta, chúng ta không bao giờ gọi các phương thức trực tiếp trên các đối tượng thuộc loại `T` . Trong ví dụ này, `T` và `TypeVar` là những kiểu dữ liệu không được xác định rõ ràng; vì vậy, các công cụ kiểm tra kiểu dữ liệu như `mypy` không thể xác định được các thuộc tính hay phương thức nào của các đối tượng `T` . Nếu cần truy cập vào các thuộc tính của các đối tượng `T` , thì `T` cần được định nghĩa bằng cách sử dụng một dạng biến thể của `TypeVar` .

Here are some examples of `TypeVar` definitions:

Dưới đây là một vài ví dụ về cách định nghĩa `TypeVar` :

Explain this code Hãy giải thích đoạn mã này. Translate into Dịch sang tiếng Việt

```
# T must be one of the types listed (int, float, complex, or str)
T = typing.TypeVar('T', int, float, complex, str)
# T must be the class MyClass or a subclass of the class MyClass
T = typing.TypeVar('T', bound=MyClass)
# T must implement __len__ to be a valid subclass of the Sized protocol
T = typing.TypeVar('T', bound=collections.abc.Sized)
```

These forms of `T` allow a generic defined on `T` to use methods from these types in `T`'s `TypeVar` definition.

Các dạng T này cho phép các đối tượng được định nghĩa trên T có thể sử dụng các phương thức thuộc các kiểu dữ liệu này trong phần định nghĩa `TypeVar` của T .

NamedTuple

The `collections.namedtuple` function simplifies the definition of class-like tuple types that support named access to the tuple elements. `NamedTuple` provides a typed version of this feature, using a class with attributes-style syntax similar to `dataclasses` (covered in **"Data Classes"**). Here's a `NamedTuple` with four elements, with names, types, and optional default values:

Hàm `collections.namedtuple` giúp đơn giản hóa cách định nghĩa các kiểu dữ liệu dạng tuple, cho phép truy cập vào các phần tử của tuple bằng tên của chúng. Hàm `NamedTuple` cung cấp phiên bản có kiểu dữ liệu rõ ràng của tính năng này; nó sử dụng cú pháp tương tự như các lớp trong `dataclasses` (được trình bày trong mục "Các lớp dữ liệu"). Dưới đây là ví dụ về một đối tượng thuộc kiểu `NamedTuple` , gồm bốn phần tử, kèm theo tên, kiểu dữ liệu và giá trị mặc định nếu có:

Explain this code Hãy giải thích đoạn mã này. Translate into Dịch sang tiếng Việt

```
class HouseListingTuple(typing.NamedTuple):
    address: str
    list_price: int
    square_footage: int = 0
    condition: str = 'Good'
```

`NamedTuple` classes generate a default constructor, accepting positional or named arguments for each named field:

Các lớp `NamedTuple` đều có một hàm khởi tạo mặc định; hàm này chấp nhận các tham số được truyền vào theo vị trí hoặc theo tên của từng trường trong lớp.

Explain this code Hãy giải thích đoạn mã này. Translate into Dịch sang tiếng Việt

```
listing1 = HouseListingTuple(
    address='123 Main',
    list_price=100_000,
    square_footage=2400,
    condition='Good',
)

print(listing1.address)  # prints: 123 Main
print(type(listing1))    # prints: <class 'HouseListingTuple'>
```

Attempting to create a tuple with too few elements raises a runtime error:

Nỗ lực tạo một tuple chứa quá ít phần tử sẽ gây ra lỗi trong quá trình thực thi chương trình.

Explain this code Hãy giải thích đoạn mã này. Translate into Dịch sang tiếng Việt

```
listing2 = HouseListingTuple(
    '123 Main',
)
# raises a runtime error: TypeError: HouseListingTuple.__new__()
# missing 1 required positional argument: 'list_price'
```

TypedDict

3.8+ Python `dict` variables are often difficult to decipher in legacy codebases, because `dict`s are used in two ways: as collections of key/value pairs (such as a mapping from user ID to username), and records mapping known field names to values. It is usually easy to see that a function argument is to be passed as a `dict`, but the actual keys and value types are dependent on the code that may call that function. Beyond simply defining that a `dict` may be a mapping of `str` values to `int` values, as in `dict[str, int]`, a `TypedDict` defines the expected keys and the types of each corresponding value. The following example defines a `TypedDict` version of the previous house listing type (note that `TypedDict` definitions do not accept default value definitions):

3.8+ biến Python trong các mã nguồn cũ có thể khó hiểu, vì các biến được sử dụng theo hai cách: như một tập hợp các cặp khóa/giá trị (ví dụ: ánh xạ từ ID người dùng đến tên người dùng), và ánh xạ các tên trường đã biết đến các giá trị. Thông thường, việc nhận biết rằng một tham số hàm cần được truyền dưới dạng một biến, nhưng các loại khóa và giá trị thực tế phụ thuộc vào mã gọi hàm đó. Ngoài việc chỉ định rằng một biến có thể là một ánh xạ giữa các giá trị, như trong ví dụ, một biến định nghĩa các khóa và loại dữ liệu của mỗi giá trị tương ứng. Ví dụ sau định nghĩa một biến tương tự như kiểu danh sách nhà trước đó (lưu ý rằng định nghĩa biến không chấp nhận định nghĩa giá trị mặc định):

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
class HouseListingDict(typing.TypedDict):
    address: str
    list_price: int
    square_footage: int
    condition: str
```

`TypedDict` classes generate a default constructor, accepting named arguments for each defined key:

Các lớp `TypedDict` đều có một hàm khởi tạo mặc định; hàm này chấp nhận các tham số có tên tương ứng với từng khóa được định nghĩa trong lớp.

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
listing1 = HouseListingDict(
    address='123 Main',
    list_price=100_000,
    square_footage=2400,
    condition='Good',
)

print(listing1['address'])  # prints 123 Main
print(type(listing1))  # prints <class 'dict'>

listing2 = HouseListingDict(
    address='124 Main',
    list_price=110_000,
)
```

Unlike the `NamedTuple` example, `listing2` will not raise a runtime error, simply creating a `dict` with just the given keys. However, `mypy` will flag `listing2` as a type error with the message:

Khác với ví dụ `NamedTuple` , `listing2` sẽ không gây ra lỗi trong quá trình thực thi chương trình. Thay vào đó, `listing2` chỉ tạo ra một đối tượng `dict` chứa các khóa đã được chỉ định. Tuy nhiên, `mypy` sẽ coi `listing2` là một lỗi kiểu dữ liệu, và hiển thị thông báo tương ứng.

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
error: Missing keys ("square_footage", "condition") for TypedDict
"HouseListing"
```

To indicate to the type checker that some keys may be omitted (but to still validate those that are given), add `total=False` to the class declaration:

Để báo cho bộ kiểm tra kiểu dữ liệu biết rằng một số khóa có thể bị bỏ qua (nhưng những khóa còn lại vẫn phải được kiểm tra), hãy thêm `total=False` vào phần khai báo lớp.

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
class HouseListing(typing.TypedDict, total=False):
    # ...
```

3.11+ Individual fields can also use the `Required` or `NotRequired` type annotations to explicitly mark them as required or optional:

3.11+ Các trường dữ liệu riêng lẻ cũng có thể sử dụng các ghi chú kiểu `Required` hoặc `NotRequired` để chỉ rõ rằng trường đó là bắt buộc hay tùy chọn.

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
class HouseListing(typing.TypedDict):
    address: typing.Required[str]
    list_price: int
    square_footage: typing.NotRequired[int]
    condition: str
```

`TypedDict` can be used to define a generic type, too:

`TypedDict` cũng có thể được sử dụng để định nghĩa một kiểu dữ liệu tổng quát.

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
T = typing.TypeVar('T')

class Node(typing.TypedDict, typing.Generic[T]):
```

```
    label: T
    neighbors: list[T]

n = Node(label='Acme', neighbors=['anvil', 'magnet', 'bird seed'])
```

DO NOT USE THE LEGACY TYPEDDICT(NAME, **FIELDS) FORMAT

KHÔNG SỬ DỤNG ĐỊNH DẠNG CŨ: TYPEDDICT (TÊN, **CÁC TRƯỜNG)**

To support backporting to older versions of Python, the initial release of `TypedDict` also let you use a syntax similar to that for `namedtuple`, such as:

Để hỗ trợ việc áp dụng các cải tiến lên các phiên bản Python cũ hơn, phiên bản đầu tiên của `TypedDict` cũng cho phép người dùng sử dụng cú pháp tương tự như khi sử dụng `namedtuple` . Ví dụ:

Explain this code Hãy giải thích đoạn mã này.

Translate into Dịch sang tiếng Việ▾

```
HouseListing = TypedDict('HouseListing',
                         address=str,
                         list_price=int,
                         square_footage=int,
                         condition=str)
```

or: Hoặc:

Explain this code Hãy giải thích đoạn mã này.

Translate into Dịch sang tiếng Việ▾

```
HouseListing = TypedDict('HouseListing',
                         {'address': str,
                          'list_price': int,
                          'square_footage': int,
                          'condition': str})
```

These forms are deprecated in Python 3.11, and are planned to be removed in Python 3.13.

Những kiểu dữ liệu này đã lỗi thời trong Python 3.11, và dự kiến sẽ bị loại bỏ hoàn toàn trong Python 3.13.

Note that `TypedDict` does not actually define a new type. Classes created by inheriting from `TypedDict` actually serve as `dict` factories, such that instances created from them *are* `dict`s. Reusing the previous code snippet defining the `Node` class, we can see this using the `type` built-in function:

Lưu ý rằng `TypedDict` thực chất không định nghĩa một kiểu dữ liệu mới nào cả. Các lớp được tạo ra bằng cách kế thừa từ `TypedDict` thực chất đóng vai trò như những "công cụ tạo ra" các đối tượng thuộc kiểu `dict` . Những đối tượng được tạo ra từ các lớp này chính là các đối tượng thuộc kiểu `dict` . Bằng cách sử dụng đoạn mã đã được định nghĩa trước đó để tạo lớp `Node` , chúng ta có thể kiểm tra điều này bằng cách sử dụng hàm tích hợp sẵn `type` .

Explain this code Hãy giải thích đoạn mã này.

Translate into Dịch sang tiếng Việ▾

```
n = Node(label='Acme', neighbors=['anvil', 'magnet', 'bird seed'])
print(type(n))              # prints: <class 'dict'>
print(type(n) is dict)      # prints: True
```

There is no special runtime conversion or initialization when using `TypedDict`; the benefits of `TypedDict` are those of static type checking and self documentation, which naturally accrue from using type annotations.

Khi sử dụng `TypedDict` , không cần phải thực hiện bất kỳ quy trình chuyển đổi dữ liệu hay khởi tạo nào đặc biệt. Những lợi ích của việc sử dụng `TypedDict` chính là khả năng kiểm tra kiểu dữ liệu một cách tự động và tính năng ghi chú thông tin về kiểu dữ liệu. Những lợi ích này là kết quả tự nhiên của việc sử dụng các ghi chú về kiểu dữ liệu.

WHICH SHOULD YOU USE, NAMEDTUPLE OR TYPEDDICT?

BẠN NÊN SỬ DỤNG CÁI NÀO: NAMEDTUPLE HAY TYPEDDICT?

The two data types appear similar in terms of their supported features, but there are significant differences that should help you determine which one to use.

Hai loại dữ liệu này có vẻ tương tự nhau về các tính năng mà chúng hỗ trợ. Tuy nhiên, vẫn có những điểm khác biệt quan trọng mà bạn cần lưu ý để quyết định nên sử dụng loại nào.

`NamedTuple`s are immutable, so they can be used as dictionary keys or stored in sets, and are inherently safe to share across threads. As a `NamedTuple` object is a tuple, you can get its property values in order simply by iterating over it. However, to get the attribute names, you need to use the special `__annotations__` attribute.

`NamedTuple` là những giá trị không thể thay đổi, vì vậy chúng có thể được sử dụng làm khóa trong từ điển hoặc được lưu trữ trong tập hợp dữ liệu. Chúng cũng rất an toàn khi được chia sẻ giữa các luồng thực thi. Vì một đối tượng `NamedTuple` thuộc dạng tuple, nên bạn có thể lấy các giá trị thuộc tính của nó một cách dễ dàng bằng cách duyệt qua chúng. Tuy nhiên, để lấy tên các thuộc tính, bạn cần sử dụng thuộc tính đặc biệt `__annotations__` .

Since classes created with `TypedDict` are actually `dict` factories, instances created from them are `dict`s, with all the behavior and attributes of `dict`s. They are mutable, so their values can be updated without creating a new container instance, and they support all the `dict` methods, such as `keys`, `values`, and `items`. They are also easily serialized using JSON or `pickle`. However, being mutable, they cannot be used as keys in another `dict`, nor can they be stored in a `set`.

Vì các lớp được tạo bằng `TypedDict` thực tế là các nhà máy `dict` , các đối tượng được tạo từ chúng là `dict` , với tất cả các hành vi và thuộc tính của `dict` . Chúng có thể thay đổi, nghĩa là giá trị của chúng có thể được cập nhật mà không cần tạo một phiên bản mới, và chúng hỗ trợ tất cả các phương thức `dict` , chẳng hạn như `keys` , `values` và `items` . Chúng cũng có thể dễ dàng được định dạng bằng JSON hoặc `pickle` . Tuy nhiên, do có khả năng thay đổi, chúng không thể được sử dụng làm khóa trong một `dict` , cũng như không thể được lưu trữ trong một `set` .

`TypedDict`s are more lenient than `NamedTuple`s about missing keys. When a key is omitted when constructing a `TypedDict`, there is no error (though you will get a type-check warning from the static type checker). On the other hand, if an attribute is omitted when constructing a `NamedTuple`, this will raise a runtime `TypeError`.

`TypedDict` thì linh hoạt hơn `NamedTuple` trong trường hợp có chữ cái nào đó bị bỏ qua khi tạo ra `TypedDict` . Khi một chữ cái bị bỏ qua, sẽ không xảy ra lỗi nào cả (mặc dù bạn sẽ nhận được cảnh báo từ công cụ kiểm tra kiểu dữ liệu tĩnh). Ngược lại, nếu một thuộc tính nào đó bị bỏ qua khi tạo ra `NamedTuple` , thì sẽ xảy ra lỗi trong quá trình thực thi chương trình.

In short, there is no across-the-board rule for when to use a `NamedTuple` versus a `TypedDict`. Consider these alternative behaviors and how they relate to your program and its use of these data objects when deciding between a `NamedTuple` and a `TypedDict`—and don't forget the other, often preferable, alternative of using a `dataclass` (covered in **"Data Classes"**) instead!

Nói ngắn gọn, không có quy tắc cố định nào để xác định lúc nào nên sử dụng `NamedTuple` thay vì `TypedDict` . Hãy xem xét các phương án khác nhau và cách chúng liên quan đến chương trình của bạn cũng như cách chương trình sử dụng các đối tượng dữ liệu này. Đừng quên phương án khác, thường là phương án tốt hơn, đó là sử dụng `dataclass` thay thế (điều này sẽ được trình bày chi tiết trong mục "Các lớp dữ liệu").

TypeAlias Kiểu suy luận

3.10+ Defining a simple type alias can be misinterpreted as assigning a class to a variable. For instance, here we define a type for record identifiers in a database:

3.10+ Việc định nghĩa một biệt danh cho kiểu dữ liệu đơn giản có thể bị hiểu nhầm là việc gán một lớp vào biến. Ví dụ, ở đây chúng ta định nghĩa một kiểu dữ liệu dùng để đại diện cho các tham số trong cơ sở dữ liệu:

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt▾

```
Identifier = int
```

To clarify that this statement is intended to define a custom type name for the purposes of type checking, use `TypeAlias`:

Để làm rõ rằng phát biểu này nhằm xác định tên của một kiểu dữ liệu tùy chỉnh, nhằm mục đích kiểm tra kiểu dữ liệu, hãy sử dụng `TypeAlias` .

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt▾

```
Identifier: TypeAlias = int
```

`TypeAlias` is also useful when defining an alias for a type that is not yet defined, and so referenced as a string value:

`TypeAlias` cũng rất hữu ích khi cần đặt tên viết tắt cho một kiểu dữ liệu chưa được định nghĩa trước đó; lúc đó, kiểu dữ liệu đó có thể được tham chiếu dưới dạng giá trị chuỗi.

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt▾

```
# Python will treat this like a standard str assignment
TBDType = 'ClassNotDefinedYet'

# indicates that this is actually a forward reference to a class
TBDType: TypeAlias = 'ClassNotDefinedYet'
```

`TypeAlias` types may only be defined at module scope. Custom types defined using `TypeAlias` are interchangeable with the target type. Contrast `TypeAlias` (which does not create a new type, just gives a new name for an existing one) with `NewType`, covered in the following section, which does create a new type.

Các kiểu dữ liệu `TypeAlias` chỉ có thể được định nghĩa trong phạm vi của từng mô-đun. Các kiểu dữ liệu được định nghĩa bằng `TypeAlias` có thể được sử dụng thay thế cho kiểu dữ liệu tương ứng trong chương trình. Hãy so sánh `TypeAlias` (không tạo ra kiểu dữ liệu mới, mà chỉ đặt tên mới cho kiểu dữ liệu đã tồn tại) với NewType – kiểu dữ liệu được tạo ra bằng NewType. Chi tiết về NewType sẽ được trình bày ở phần sau.

NewType

`NewType` allows you to define application-specific subtypes, to avoid confusion that might result from using the same type for different variables. If your program uses `str` values for different types of data, for example, it is easy to accidentally interchange values. Suppose you have a program that models employees in departments. The following type declaration is not sufficiently descriptive—which is the key and which is the value?

NewType cho phép bạn định nghĩa các kiểu dữ liệu riêng biệt dành cho từng ứng dụng cụ thể, nhằm tránh sự nhầm lẫn có thể xảy ra khi sử dụng cùng một kiểu dữ liệu cho nhiều biến khác nhau. Nếu chương trình của bạn sử dụng các giá trị thuộc kiểu `str` cho nhiều loại dữ liệu khác nhau, thì rất dễ xảy ra tình trạng nhầm lẫn giữa các giá trị này. Giả sử bạn có một chương trình dùng để mô tả các nhân viên trong các bộ phận khác nhau. Đoạn khai báo kiểu dữ liệu dưới đây chưa đủ rõ ràng: Điều gì là khóa, và điều gì là giá trị?

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
employee_department_map: dict[str, str] = {}
```

Defining types for employee and department IDs makes this declaration clearer:

Việc xác định rõ các loại dữ liệu dành cho mã nhân viên và mã bộ phận sẽ giúp câu khai báo này trở nên rõ ràng hơn:

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
EmpId = typing.NewType('EmpId', str)
DeptId = typing.NewType('DeptId', str)
employee_department_map: dict[EmpId, DeptId] = {}
```

These type definitions will also allow type checkers to flag this incorrect usage:

Những định nghĩa kiểu dữ liệu này cũng giúp các công cụ kiểm tra kiểu dữ liệu phát hiện những trường hợp sử dụng sai cách.

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
def transfer_employee(empid: EmpId, to_dept: DeptId):
    # update department for employee
     employee_department_map[to_dept] = empid
```

Running `mypy` reports these errors for the line `employee_department_map[to_dept] = empid`:

Chương trình chạy mypy báo cáo các lỗi liên quan đến dòng `employee_department_map[to_dept] = empid` :

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
error: Invalid index type "DeptId" for "Dict[EmpId, DeptId]"; expected
type "EmpId"
error: Incompatible types in assignment (expression has type "EmpId",
target has type "DeptId")
```

Using `NewType` often requires you to use `typing.cast` too; for example, to create an `EmpId`, you need to cast a `str` to the `EmpId` type.

Việc sử dụng NewType thường đòi hỏi bạn phải sử dụng `typing.cast` cùng lúc. Ví dụ, để tạo ra một `EmpId` , bạn cần chuyển đổi kiểu dữ liệu của `str` thành kiểu `EmpId` .

You can also use `NewType` to indicate the desired implementation type for an application-specific type. For instance, the basic US postal zip code is five numeric digits. It is common to see this implemented using `int`, which becomes problematic with zip codes that have a leading `0`. To indicate that zip codes should be implemented using `str`, your code can define this type-checking type:

Bạn cũng có thể sử dụng NewType để chỉ định kiểu dữ liệu cần được sử dụng cho các loại dữ liệu riêng biệt trong ứng dụng. Ví dụ, mã bưu chính ở Hoa Kỳ gồm 5 chữ số. Thông thường, người ta sử dụng `int` để biểu diễn kiểu dữ liệu này. Tuy nhiên, cách này gây ra vấn đề khi mã bưu chính bắt đầu bằng `0` . Để chỉ định rằng mã bưu chính nên được biểu diễn bằng `str` , bạn có thể định nghĩa kiểu dữ liệu này trong mã nguồn của mình.

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việ▾

```
ZipCode = typing.NewType("ZipCode", str)
```

Annotating variables and function arguments using `ZipCode` will help flag incorrect uses of `int` for zip code values.

Việc ghi chú các biến và tham số hàm bằng ký hiệu `ZipCode` sẽ giúp nhận diện những trường hợp sử dụng sai ký hiệu `int` trong các giá trị mã bưu điện.

Using Type Annotations at Runtime

Sử dụng ghi chú kiểu dữ liệu tại thời gian thực

Function and class variable annotations can be introspected by accessing the function or class's `__annotations__` attribute (although a **better practice** is to instead call `inspect.get_annotations()`):

Các ghi chú liên quan đến các biến hàm và biến lớp có thể được xem xét bằng cách truy cập vào thuộc tính `__annotations__` của hàm hoặc lớp đó (mặc dù cách tốt hơn là nên sử dụng thuộc tính `inspect.get_annotations()`).

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt▾

```
>>> def f(a:list[str], b) -> int:
...     pass
...
>>> f.__annotations__
```

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt▾

```
{'a': list[str], 'return': <class 'int'>}
```

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt▾

```
>>> class Customer:
...     name: str
...     reward_points: int = 0
...
>>> Customer.__annotations__
```

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt▾

```
{'name': <class 'str'>, 'reward_points': <class 'int'>}
```

This feature is used by third-party packages such as `pydantic` and FastAPI to provide extra code generation and validation capabilities.

Tính năng này được các công cụ của bên thứ ba như `pydantic` và FastAPI sử dụng để tạo ra các đoạn mã bổ sung và thực hiện các thao tác kiểm tra mã một cách hiệu quả.

3.8+ To define your own custom protocol class that supports runtime checking with `issubclass` and `isinstance`, define that class as a subclass of `typing.Protocol`, with empty method definitions for the required protocol methods, and decorate the class with `@runtime_checkable` (covered in **Table 5-6**). If you *don't* decorate it with `@runtime_checkable`, you're still defining a `Protocol` that's quite usable for static type checking, but it won't be runtime-checkable with `issubclass` and `isinstance`.

3.8+ Để định nghĩa lớp giao thức tùy chỉnh của riêng mình, nhằm hỗ trợ việc kiểm tra trong quá trình thực thi chương trình bằng các ký hiệu `issubclass` và `isinstance` , hãy định nghĩa lớp đó như là lớp con của `typing.Protocol` . Các phương thức cần thiết của giao thức này nên được định nghĩa thành các phương thức trống. Đồng thời, hãy sử dụng ký hiệu `@runtime_checkable` để đánh dấu lớp đó (chi tiết được trình bày trong Bảng 5-6). Nếu không sử dụng ký hiệu `@runtime_checkable` , bạn vẫn sẽ tạo ra một lớp `Protocol` có thể được sử dụng để kiểm tra kiểu dữ liệu một cách tĩnh; tuy nhiên, lớp này sẽ không thể được kiểm tra trong quá trình thực thi chương trình bằng các ký hiệu `issubclass` và `isinstance` .

For example, we could define a protocol that indicates that a class implements the `update` and `undo` methods as follows (the Python `Ellipsis`, `...`, is a convenient syntax for indicating an empty method definition):

Ví dụ, chúng ta có thể định nghĩa một giao thức như sau: một lớp nào đó phải triển khai các phương thức `update` và `undo` . Cách diễn đạt này tương tự như cú pháp `Ellipsis` và `...` trong Python – những cú pháp này được sử dụng để chỉ việc định nghĩa một phương thức trống.

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt▾

```
T = typing.TypeVar('T')

@typing.runtime_checkable
class SupportsUpdateUndo(typing.Protocol):
    def update(self, *args: T) -> None:
        ...
    def undo(self) -> None:
        ...
```

Without making any changes to the inheritance path of `Accumulator` (defined in **"Generics and TypeVars"**), it now satisfies runtime type checks with `SupportsUpdateUndo`:

Không cần thực hiện bất kỳ thay đổi nào đối với cách thừa kế của `Accumulator` (được định nghĩa trong mục "Generics and TypeVars"), giờ đây `Accumulator` vẫn đáp ứng các yêu cầu kiểm tra kiểu dữ liệu trong quá trình thực thi chương trình, khi được sử dụng cùng với `SupportsUpdateUndo` .

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt ▾

```
>>> issubclass(Accumulator, SupportsUpdateUndo)
```

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt ▾

```
True
```

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt ▾

```
>>> isinstance(acc, SupportsUpdateUndo)
```

Explain this code Hãy giải thích đoạn mã này. **Translate into** Dịch sang tiếng Việt ▾

```
True
```

In addition, any other class that implements `update` and `undo` methods will now qualify as a `SupportsUpdateUndo` "subclass."

Ngoài ra, bất kỳ lớp nào khác cũng có thể được coi là "lớp con" của `SupportsUpdateUndo` , miễn là lớp đó triển khai các phương thức `update` và `undo` .

How to Add Type Annotations to Your Code

Làm thế nào để thêm các ghi chú về kiểu dữ liệu vào mã nguồn của bạn

Having seen some of the features and capabilities provided by using type annotations, you may be wondering about the best way to get started. This section describes a few scenarios and approaches to adding type annotations.

Sau khi tìm hiểu một số tính năng và khả năng mà việc sử dụng chú thích kiểu dữ liệu mang lại, có lẽ bạn đang băn khoăn về cách tốt nhất để bắt đầu sử dụng chú thích kiểu dữ liệu này. Phần này sẽ trình bày một vài cách thức và kịch bản cụ thể để áp dụng chú thích kiểu dữ liệu.

Adding Type Annotations to New Code

Thêm ghi chú về kiểu dữ liệu vào mã nguồn mới

When you start writing a short Python script, adding type annotations may seem like an unnecessary extra burden. As a spinoff of the **Two Pizza Rule**, we suggest the Two Function Rule: as soon as your script contains two functions or methods, go back and add type annotations to the method signatures, and any shared variables or types as necessary. Use `TypedDict` to annotate any `dict` structures that are used in place of classes, so that `dict` keys get clearly defined up front or get documented as you go; use `NamedTuples` (or `dataclasses`: some of this book's authors *strongly* prefer the latter option) to define the specific attributes needed for those data "bundles."

Khi bắt đầu viết một đoạn mã Python ngắn, việc thêm các ghi chú về kiểu dữ liệu có vẻ như là một thao tác không cần thiết. Dựa trên nguyên tắc "Two Pizza Rule", chúng tôi đề xuất nguyên tắc "Two Function Rule": ngay khi đoạn mã của bạn chứa hai hàm hoặc phương thức trở lên, hãy thêm các ghi chú về kiểu dữ liệu vào các định nghĩa của các hàm đó. Ngoài ra, hãy thêm các ghi chú về kiểu dữ liệu cho các biến được chia sẻ giữa các hàm nếu cần thiết. Hãy sử dụng ký hiệu `TypedDict` để ghi chú các cấu trúc dữ liệu được sử dụng thay thế cho các lớp; nhờ đó, các thuộc tính của các cấu trúc dữ liệu này sẽ được xác định rõ ràng ngay từ đầu hoặc được ghi chú lại trong quá trình viết mã. Hãy sử dụng ký hiệu `NamedTuple` (hoặc `dataclass` : một số tác giả trong cuốn sách này thích sử dụng ký hiệu `dataclass` hơn) để xác định các thuộc tính cụ thể cần thiết cho các cấu trúc dữ liệu đó.

If you are beginning a major project with many modules and classes, then you should definitely use type annotations from the beginning. They can easily make you more productive, as they help avoid common naming and typing mistakes and ensure you get more fully supported autocompletion while working in your IDE. This is even more important on projects with multiple developers: having documented types helps tell everyone on the team the expectations for types and values to be used across the project. Capturing these types in the code itself makes them immediately accessible and visible during development, much more so than separate documentation or specifications.

Nếu bạn đang bắt đầu một dự án lớn, bao gồm nhiều thành phần và lớp khác nhau, thì chắc chắn bạn nên sử dụng các ghi chú về kiểu dữ liệu ngay từ đầu. Việc này sẽ giúp bạn làm việc hiệu quả hơn, vì nó giúp tránh những lỗi thường gặp trong việc đặt tên và nhập dữ liệu. Ngoài ra, việc sử dụng các ghi chú về kiểu dữ liệu còn giúp bạn tận dụng tối đa tính năng tự động hoàn thành câu lệnh trong môi trường phát triển IDE. Điều này càng quan trọng trong các dự án có nhiều lập trình viên tham gia: việc ghi chú rõ ràng về kiểu dữ liệu sẽ giúp mọi người trong nhóm hiểu rõ những yêu cầu liên quan đến kiểu dữ liệu và giá trị dữ liệu được sử dụng trong toàn bộ dự án. Việc ghi chú kiểu dữ liệu ngay trong mã nguồn sẽ giúp mọi người dễ dàng tiếp cận và hiểu rõ thông tin về kiểu dữ liệu trong quá trình phát triển dự án, điều mà các tài liệu hoặc yêu cầu kỹ thuật riêng lẻ khó có thể thực hiện được.

If you are developing a library to be shared across projects, then you should also use type annotations from the very start, most likely paralleling the function signatures in your API design. Having type annotations in a library will make life easier for your client developers, as all modern IDEs include type annotation plug-ins to support static type checking and function autocompletion and documentation. They will also help you when writing your unit tests, since you will benefit from the same rich IDE support.

Nếu bạn đang phát triển một thư viện để chia sẻ giữa các dự án khác nhau, thì bạn nên sử dụng các ghi chú về kiểu dữ liệu ngay từ đầu. Cách làm này tương tự như cách ghi chú các thông số của các hàm trong thiết kế API của bạn. Việc sử dụng các ghi chú về kiểu dữ liệu sẽ giúp ích rất nhiều cho các lập trình viên sử dụng thư viện của bạn. Tất cả các công cụ phát triển phần mềm hiện đại đều có các tiện ích mở rộng để hỗ trợ việc kiểm tra kiểu dữ liệu, tự động hoàn thành các câu lệnh và tạo tài liệu hướng dẫn sử dụng. Những tiện ích này cũng sẽ hỗ trợ bạn rất nhiều khi viết các bài kiểm thử đơn vị.

For any of these projects, add a type-checking utility to your pre-commit hooks, so that you stay ahead of any type infractions that might creep into your new codebase. This way you can fix them as they occur, instead of waiting until you do a large commit and finding that you have made some fundamental typing errors in multiple places.

Đối với bất kỳ dự án nào trong số này, hãy thêm công cụ kiểm tra kiểu dữ liệu vào các chương trình xử lý các thao tác gửi mã nguồn lên hệ thống lưu trữ. Điều này sẽ giúp bạn phát hiện và sửa chữa các lỗi liên quan đến kiểu dữ liệu ngay khi chúng xảy ra, thay vì phải chờ đến lúc gửi toàn bộ mã nguồn lên hệ thống lưu trữ, rồi mới phát hiện ra rằng có nhiều lỗi về kiểu dữ liệu ở nhiều nơi khác nhau.

Adding Type Annotations to Existing Code (Gradual Typing)

Thêm ghi chú về kiểu dữ liệu vào mã nguồn hiện có (Việc xác định kiểu dữ liệu một cách từ từ)

Several companies that have run projects to apply type annotations to large existing codebases recommend an incremental approach, referred to as *gradual typing*. With gradual typing, you can work through your codebase in a stepwise manner, adding and validating type annotations a few classes or modules at a time.

Một số công ty đã triển khai các dự án nhằm áp dụng các ghi chú về kiểu dữ liệu vào các bộ mã nguồn lớn hiện có. Những công ty này đề xuất cách tiếp cận từng bước một, hay còn gọi là phương pháp áp dụng ghi chú về kiểu dữ liệu một cách từ từ. Với phương pháp này, bạn có thể xử lý từng phần của bộ mã nguồn một cách từ từ, bổ sung và kiểm tra các ghi chú về kiểu dữ liệu cho từng lớp hoặc mô-đun riêng lẻ.

Some utilities, like `mypy`, will let you add type annotations function by function. `mypy`, by default, skips functions without typed signatures, so you can methodically go through your codebase a few functions at a time. This incremental process allows you to focus your efforts on individual parts of the code, as opposed to adding type annotations everywhere and then trying to sort out an avalanche of type-checker errors.

Một số công cụ hỗ trợ lập trình, như `mypy` , cho phép người dùng thêm các ghi chú về kiểu dữ liệu cho từng hàm một. Theo mặc định, `mypy` sẽ bỏ qua những hàm không có ghi chú về kiểu dữ liệu. Nhờ vậy, người dùng có thể kiểm tra từng phần của mã nguồn một cách có hệ thống. Cách tiếp cận này giúp người dùng tập trung vào từng phần cụ thể của mã nguồn, thay vì phải thêm ghi chú về kiểu dữ liệu ở khắp mọi nơi, rồi sau đó cố gắng giải quyết hàng loạt lỗi do bộ kiểm tra kiểu dữ liệu gây ra.

Some recommended approaches are:

Một số phương pháp được khuyến nghị bao gồm:

- Identify your most heavily used modules, and begin adding types to them, a method at a time. (These could be core application class modules, or widely shared utility modules.)

  Hãy xác định những mô-đun được sử dụng thường xuyên nhất, sau đó bắt đầu thêm các kiểu dữ liệu vào chúng, từng mô-đun một. (Đó có thể là các mô-đun thuộc lớp cốt lõi của ứng dụng, hoặc các mô-đun hỗ trợ chung được sử dụng rộng rãi.)

- Annotate a few methods at a time, so that type-checking issues get raised and resolved gradually.

  Hãy ghi chú từng phương pháp một, như vậy các vấn đề liên quan đến việc kiểm tra kiểu dữ liệu sẽ được phát hiện và giải quyết dần dần.

- Use `pytype` or `pyre` inference to generate initial *.pyi* stub files (discussed in the following section). Then, steadily migrate types from the *.pyi* files, either manually or using automation such as `pytype`'s `merge_pyi` utility.

  Hãy sử dụng các công cụ `pytype` hoặc `pyre` để tạo các tệp tin .pyi ban đầu (chi tiết sẽ được trình bày ở phần sau). Sau đó, hãy chuyển các thông tin về kiểu dữ liệu từ các tệp tin .pyi sang các định dạng khác, có thể thực hiện việc này một cách thủ công hoặc bằng cách sử dụng các công cụ tự động hóa như `pytype` và `merge_pyi` .

- Begin using type checkers in a lenient default mode, so that most code is skipped and you can focus attention on specific files. As work progresses, shift to a stricter mode so that remaining items are made more prominent, and files that have been annotated do not regress by taking on new nonannotated code.

  Hãy bắt đầu sử dụng các công cụ kiểm tra kiểu dữ liệu ở chế độ mặc định, để hầu hết các đoạn mã không cần được kiểm tra. Nhờ đó, bạn có thể tập trung vào các tệp tin cụ thể. Khi công việc tiến triển, hãy chuyển sang chế độ kiểm tra nghiêm ngặt hơn, để những đoạn mã cần kiểm tra được nổi bật lên. Đồng thời, các tệp tin đã được kiểm tra sẽ không bị ảnh hưởng bởi các đoạn mã mới chưa được kiểm tra.

Using .pyi Stub Files

Sử dụng các tệp stub có định dạng .pyi

Sometimes you don’t have access to Python type annotations. For example, you might be using a library that does not have type annotations, or using a module whose functions are implemented in C.

Đôi khi, bạn không thể sử dụng các ghi chú về kiểu dữ liệu trong Python. Ví dụ, bạn có thể đang sử dụng một thư viện không hỗ trợ các ghi chú về kiểu dữ liệu, hoặc đang sử dụng một mô-đun mà các hàm trong đó được viết bằng ngôn ngữ C.

In these cases, you can use separate *.pyi* stub files containing just the related type annotations. Several of the type checkers mentioned at the beginning of this chapter can generate these stub files. You can download stub files for popular Python libraries, as well as the Python standard library itself, from the **typeshed repository**. You can maintain stub files from the Python source, or, using merging utilities available in some of the type checkers, integrate them back into the original Python source.

Trong những trường hợp như vậy, bạn có thể sử dụng các tệp tin .pyi chứa chỉ các ghi chú về kiểu dữ liệu liên quan. Nhiều công cụ kiểm tra kiểu dữ liệu được đề cập ở đầu chương này đều có thể tạo ra các tệp tin này. Bạn có thể tải về các tệp tin .pyi dành cho các thư viện Python phổ biến, cũng như chính thư viện tiêu chuẩn của Python, từ kho lưu trữ `typeshed` . Bạn có thể tự quản lý các tệp tin .pyi này từ mã nguồn Python, hoặc sử dụng các công cụ hợp nhất dữ liệu có sẵn trong một số công cụ kiểm tra kiểu dữ liệu để tích hợp chúng trở lại vào mã nguồn Python gốc.

DO TYPE ANNOTATIONS GET IN THE WAY OF CODING?

LIỆU CÁC GHI CHÚ KIỂU NÀY CÓ CẢN TRỞ QUÁ TRÌNH LẬP TRÌNH HAY KHÔNG?

Type annotations carry some stigma, especially for those who have worked with Python for many years and are used to taking full advantage of Python’s adaptive nature. Flexible method signatures like that of the built-in function `max`, which can take a single argument containing a sequence of values or multiple arguments containing the values to be maximized, have been cited as being **especially challenging to type-annotate**. Is this the fault of the code? Of typing? Of Python itself? Each of these explanations is possible.

Việc ghi chú kiểu dữ liệu của các biến trong mã nguồn thường gây ra không ít khó khăn, đặc biệt đối với những người đã làm việc với Python trong nhiều năm và quen với việc tận dụng tối đa những đặc tính linh hoạt của ngôn ngữ này. Các hàm có cấu trúc linh hoạt như hàm tích hợp `max` , vốn có thể nhận một đối số chứa một dãy các giá trị, hoặc nhiều đối số chứa các giá trị cần được xử lý, thường rất khó để ghi chú kiểu dữ liệu một cách chính xác. Liệu đây có phải là lỗi của chính đoạn mã? Hay do cách ghi chú kiểu dữ liệu? Hoặc có lẽ do bản chất của chính ngôn ngữ Python? Mọi giải thích trên đều có thể xảy ra.

In general, typing fosters a degree of formalism and discipline that can be more confining than the historical Python philosophy of “coding by and for consenting adults.” Moving forward, we may find that the flexibility of style in older Python code is not wholly conducive to long-term use, reuse, and maintenance by those who are not the original code authors. As a recent PyCon presenter **suggested**, “Ugly type annotations hint at ugly code.” (However, it may sometimes be the case, like for `max`, that it’s the typing system that’s not expressive enough.)

Nhìn chung, việc sử dụng các quy tắc kiểu dữ liệu một cách chặt chẽ sẽ tạo ra một mức độ tính hình thức và kỷ luật nhất định; điều này có thể gây bất lợi so với triết lý phát triển Python truyền thống, đó là “lập trình bởi và vì những người trưởng thành, những người tự nguyện tham gia vào quá trình lập trình”. Trong tương lai, có lẽ chúng ta sẽ nhận ra rằng sự linh hoạt trong cách viết mã trong các phiên bản Python cũ không hề thuận lợi cho việc sử dụng, tái sử dụng và bảo trì mã nguồn trong thời gian dài, đặc biệt là đối với những người không phải là tác giả ban đầu của đoạn mã đó. Như một diễn giả tại PyCon gần đây đã nói: “Những ghi chú về kiểu dữ liệu kém rõ ràng thường là dấu hiệu của mã nguồn kém chất lượng.” (Tuy nhiên, đôi khi cũng có trường hợp, như trong trường hợp của `max` , khiến cho hệ thống xác định kiểu dữ liệu trở nên kém hiệu quả.)

You can take the level of typing difficulty as an indicator of your method design. If your methods require multiple `Union` definitions, or multiple overrides for the same method using different argument types, perhaps your design is too flexible across multiple calling styles. You may be overdoing the flexibility of your API because Python allows it, but that might not always be a good idea in the long run. After all, as the **Zen of Python** says, “There should be one—and preferably only one—obvious way to do it.” Maybe that should include “only one obvious way” to call your API!

Bạn có thể coi mức độ khó trong việc gõ chữ như một chỉ số để đánh giá cách thiết kế phương thức của mình. Nếu các phương thức của bạn yêu cầu nhiều định nghĩa khác nhau, hoặc cần được áp dụng theo nhiều cách khác nhau tùy thuộc vào loại đối số được sử dụng, có lẽ cách thiết kế của bạn quá linh hoạt. Có lẽ bạn đang làm quá mức cần thiết trong việc tăng độ linh hoạt của API, chỉ vì Python cho phép điều đó. Nhưng về lâu dài, điều này không nhất thiết là ý tưởng tốt. Như lời khuyên trong triết lý phát triển phần mềm của Python: “Nên chỉ có một cách duy nhất để thực hiện một việc gì đó.” Có lẽ điều này cũng nên được áp dụng trong cách gọi API của bạn!

Summary Tóm tắt

Python has steadily risen to prominence as a powerful language and programming ecosystem, supporting important enterprise applications. What was once a utility language for scripting and task automation has become a platform for significant and complex applications affecting millions of users, used in mission-critical and even extraterrestrial systems.[5] Adding type annotations is a significant step in developing and maintaining these systems.

Python ngày càng trở nên quan trọng như một ngôn ngữ lập trình mạnh mẽ và một nền tảng lập trình hiệu quả, hỗ trợ nhiều ứng dụng quan trọng trong doanh nghiệp. Từ một ngôn ngữ dùng để viết các chương trình tự động hóa, Python giờ đây đã trở thành nền tảng cho nhiều ứng dụng phức tạp, ảnh hưởng đến hàng triệu người dùng. Python được sử dụng trong nhiều hệ thống quan trọng, thậm chí cả trong các hệ thống liên quan đến hoạt động ngoài Trái Đất. Việc thêm các ghi chú về kiểu dữ liệu là bước quan trọng trong việc phát triển và bảo trì các hệ thống này.

The **online documentation** for type annotations provides up-to-date descriptions, examples, and **best practices** as the syntax and practices for annotating types continue to evolve. The authors also recommend ***Fluent Python*, 2nd edition**, by Luciano Ramalho (O'Reilly), especially Chapters 8 and 15, which deal specifically with Python type annotations.

Tài liệu hướng dẫn về cách ghi chú các kiểu dữ liệu trực tuyến cung cấp những mô tả, ví dụ và khuyến nghị hữu ích nhất, bởi vì cú pháp và cách thức ghi chú các kiểu dữ liệu liên tục thay đổi theo thời gian. Các tác giả cũng đề xuất cuốn sách "Fluent Python, 2nd edition" của Luciano Ramalho (O'Reilly). Đặc biệt, các chương 8 và 15 trong cuốn sách này trình bày chi tiết về cách ghi chú các kiểu dữ liệu trong Python.